Semantiche di esecuzione e architetture parallele per una formulazione continua della Particle Swarm Optimization

Giovanni Giuseppe Iacuzzo[1,*]

[1]Dipartimento di Ingegneria e Architettura, Università degli Studi di Enna "Kore", Enna, Italia

[*]Autore corrispondente: giovanni.iacuzzo@unikore.it

Sommario

La parallelizzazione della Particle Swarm Optimization (PSO) non è una scelta puramente implementativa: a seconda del punto in cui si introducono sincronizzazione, pubblicazione degli aggiornamenti e migrazione, essa può preservare la traiettoria del metodo seriale oppure definire una dinamica di ricerca differente.In questo studio sono formulate cinque modalità di esecuzione del *Continuous Particle Swarm Optimization*, implementate in un motore CPU a memoria condivisa: seriale sincrona, master–slave sincrona, particle-parallel sincrona, asincrona event-driven e a isole. Per le due modalità sincrone parallele il contratto di aggiornamento è progettato per preservare la traiettoria seriale; per le modalità asincrona e a isole la variazione della dinamica è oggetto esplicito dell'analisi. Il software implementa sette topologie di comunicazione tra particelle. Il protocollo interno comprende sei funzioni analitiche in dimensione 30 e 100 e un test separato di strong scaling con carico sintetico. Un livello esterno confronta la CPSO con dieci baseline PSO discrete in spazio continuo su COCO BBOB, BBOB-LargeScale e CEC 2013 LSGO, a budget FE abbinato. Il manoscritto rende espliciti il propagatore analitico, i parametri, le semantiche degli esecutori, gli adattamenti delle baseline e le limitazioni statistiche e computazionali.

Parole chiave: Continuous Particle Swarm Optimization; parallelizzazione; algoritmi sincroni e asincroni; modelli a isole; equivalenza algoritmica; analisi sperimentale.

1 Introduzione

La Particle Swarm Optimization (PSO) è una metaeuristica di popolazione nella quale ciascun agente aggiorna il proprio stato combinando memoria individuale e informazione condivisa con il vicinato [1]. La parallelizzazione è naturale quando le valutazioni della funzione obiettivo sono indipendenti, ma non è neutrale rispetto all'algoritmo: una barriera di generazione, la pubblicazione immediata dei migliori personali o una migrazione tra sottopopolazioni determinano quali informazioni siano disponibili a ciascuna particella e in quale istante. Due implementazioni che eseguono lo stesso numero di valutazioni possono quindi seguire traiettorie diverse e non devono essere confrontate come se differissero soltanto per il tempo di esecuzione.

Il problema affrontato è computazionale e sperimentale: stabilire quali semantiche parallele preservino la traiettoria CPSO, quali introducano dinamiche differenti, come la struttura sociale influenzi la ricerca e in quali condizioni il parallelismo CPU riduca effettivamente il tempo di calcolo. Per motivare il confronto tra master–slave e particle-parallel è necessario considerare non soltanto il costo delle valutazioni obiettivo, ma anche quello della propagazione analitica: il propagatore contiene selezione del regime dinamico e funzioni trascendenti, mentre la mappa dei vincoli opera coordinata per coordinata. La quantità di lavoro effettivamente parallelizzabile dipende tuttavia dall'implementazione (ad esempio, i coefficienti comuni alle coordinate consentono di riutilizzare alcuni fattori trascendenti per particella) e sarà misurata.

Il primo livello dello studio riguarda gli esecutori sincroni. Tutte le particelle leggono lo stesso snapshot dei migliori personali, i calcoli indipendenti vengono eseguiti in parallelo e i risultati vengono applicati con un commit deterministico. Master–slave parallelizza le valutazioni obiettivo; particle-parallel parallelizza anche inizializzazione e propagazione locale. In presenza di obiettivi deterministici e thread-safe, stream pseudocasuali assegnati alle particelle e medesimo ordine di commit, queste modalità sono progettate per riprodurre la traiettoria del riferimento seriale

Il secondo livello riguarda modalità che cambiano la dinamica. Nell'aggiornamento asincrono event-driven ogni completamento può diventare immediatamente visibile alla popolazione; non esiste una barriera globale che imponga a tutte le particelle di osservare lo stesso stato. Nel modello a isole la popolazione è suddivisa in sottopopolazioni che evolvono localmente e si scambiano emigranti secondo una politica di migrazione. Il protocollo interno usa funzioni analitiche a costo sostanzialmente uniforme: lo studio asincrono interno isola quindi l'effetto algoritmico della pubblicazione

immediata dell'informazione sociale.

La rete sociale è un fattore sperimentale distinto dall'esecutore. Il software implementa sette famiglie: global-best, ring periodico, griglia toroidale di von Neumann, wheel, random connessa, small-world di Watts–Strogatz e scale-free di Barabási–Albert. La campagna architetturale principale usa global-best, ring e von Neumann; una campagna complementare estende l'indagine alle altre famiglie. Le proprietà di rete osservate nella dinamica PSO motivano l'inclusione di grafi irregolari, ma non dimostrano che imporre una topologia small-world o scale-free migliori necessariamente l'ottimizzazione [2]. I risultati saranno interpretati come confronti tra famiglie di grafo con profili di grado differenti, non come effetti causali della sola geometria della rete.

La qualità della ricerca viene valutata a due livelli. Il primo è un insieme interno di sei funzioni deterministiche (Sphere, Rastrigin, Rosenbrock, Ackley, Griewank e Salomon) in dimensione 30 e 100: consente confronti appaiati e controllati tra topologie e semantiche architetturali, ma è un insieme diagnostico ristretto. Quasi tutte le funzioni hanno il minimo nell'origine (Rosenbrock ha minimo nel vettore unitario); l'inizializzazione e gli attrattori combinatori possono quindi introdurre un bias verso il centro. Il secondo livello usa COCO BBOB, BBOB-LargeScale e CEC 2013 LSGO per collocare la CPSO rispetto a dieci baseline PSO discrete in spazio continuo e a testbed standardizzati [3–5]. La misura di qualità è separata da quella del parallelismo: i driver esterni invocano l'obiettivo attraverso callback seriali e non forniscono evidenza sullo scaling del motore nativo.

Il confronto algoritmico comprende dieci baseline PSO selezionate per coprire differenti meccanismi di attrazione, informazione e organizzazione della popolazione: global-best classica, inerzia, fattore di costrizione, local-best ad anello, CLPSO, SL-PSO, SPSO-2011, FIPS, Competitive Swarm Optimizer e DMS-PSO. La selezione è intenzionalmente interna alla famiglia PSO. Le formulazioni e gli adattamenti effettivamente usati nel codice sono riportati nell'Appendice A.

Le domande di ricerca sono:

- **RQ1 – Invarianza sincrona.** Master–slave e particle-parallel preservano posizione, valore e intera cronologia best-so-far del riferimento seriale sotto il contratto di aggiornamento implementato?
- **RQ2 – Scalabilità CPU.** In quali combinazioni di dimensione, popolazione e costo della valutazione le modalità sincrone parallele producono speedup ed efficienza, dopo aver verificato l'equivalenza?
- **RQ3 – Dinamiche alternative.** A parità di FE e popolazione totale, come cambiano qualità e successo quando gli aggiornamenti sono event-driven o la popolazione è organizzata in isole con migrazione?
- **RQ4 – Topologie.** Le sette famiglie di grafo producono differenze riproducibili nella qualità CPSO e nel costo di selezione sociale, limitatamente alle famiglie e ai parametri testati?
- **RQ5 – Posizionamento algoritmico.** Come si colloca la CPSO, a budget FE abbinato, rispetto alle dieci baseline PSO discrete su BBOB, BBOB-LargeScale e CEC 2013 LSGO?

Il contributo consiste nel separare esplicitamente (a) parallelismo dell'esecutore con traiettoria invariata, (b) parallelizzazione con dinamica modificata e (c) struttura della comunicazione sociale, applicando la distinzione a un unico nucleo matematico ovvero la CPSO.

2 Stato dell'arte e posizionamento

2.1 Modelli continui della PSO e posizione della CPSO di Orlando–Ricciardello

L'interpretazione della dinamica PSO come sistema del secondo ordine precede la formulazione qui studiata. Emara e Fattah hanno proposto una tecnica di ottimizzazione swarm a tempo continuo con analisi di stabilità [6]. Fernández-Martínez e García-Gonzalo hanno sviluppato il modello continuo della PSO associato all'analogia di una massa soggetta a molla e smorzamento e hanno introdotto una generalizzazione con passo temporale non unitario, analizzandone attenuazione, oscillazione e stabilità deterministica e stocastica [7–9]. Questi lavori sono essenziali per collocare storicamente la dinamica continua, ma non vengono identificati con la CPSO nel senso adottato nel presente articolo.

La rassegna di Bonyadi e Michalewicz organizza i principali aspetti della PSO in spazio continuo, inclusi stabilità, struttura della casualità, topologie, rotazione e gestione dei vincoli [10]. Grassi e Pareschi sviluppano invece una modellazione stocastica e di campo medio che collega lo swarm a equazioni di tipo McKean–Vlasov e alla consensus-based optimization [11]. Si tratta di un filone teorico complementare: non è una derivazione del propagatore analitico impiegato qui.

Nel presente manoscritto, **CPSO** è riservato alla specifica formulazione *Continuous Particle Swarm Optimization* [12]. Il suo tratto operativo distintivo, per il problema affrontato, è la propagazione analitica della dinamica locale a coefficienti congelati tra due osservazioni successive. Il nostro lavoro indaga le semantiche di esecuzione e le architetture parallele applicate a questa formulazione. Per evitare collisioni terminologiche, le altre formulazioni sono nominate con autore, anno o denominazione originale, senza riutilizzare l'acronimo CPSO.

2.2 Parallelizzazione della PSO: esecutori e semantica

La letteratura sulla parallel PSO comprende modelli master–slave, parallelismo a grana fine, aggiornamenti asincroni, accelerazione GPU e modelli a isole. Le analisi generali del parallelismo negli algoritmi evolutivi distinguono distribuzione delle valutazioni, parallelismo interno alla popolazione e decomposizione in sottopopolazioni [13, 14]; le rassegne specifiche PSO organizzano i metodi in funzione della granularità, sincronizzazione e piattaforma [15]. Le valutazioni indipendenti delle particelle sono state sfruttate in architetture globali o master–slave [16]; lavori successivi hanno studiato esplicitamente l'aggiornamento asincrono e gli effetti dei carichi di valutazione non uniformi [17], nonché implementazioni GPU [18].

PAPSO di Li et al. combina thread pool fork–join, comunicazione non bloccante e work stealing per una PSO asincrona su CPU multicore [19]. Charilogis et al. propongono modifiche e meccanismi di propagazione tra unità parallele [20]. Un'im-

Tabella 1: Motivazione sperimentale delle sette topologie implementate.

Topologia	Proprietà strutturale	Motivazione e ruolo nel disegno
Global-best	Informazione del migliore personale globale disponibile a tutti	Riferimento a massima diffusione informativa.
Ring	Vicinato locale periodico a basso grado	Riferimento classico local-best e diffusione locale dell'informazione [29].
Von Neumann	Griglia toroidale con vicini ortogonali	Topologia locale regolare bidimensionale [28].
Wheel	Un hub collegato ai nodi periferici, senza archi tra periferie	Caso di centralizzazione e dipendenza da un nodo.
Random connessa	Archi campionati casualmente con aggiunta di ponti tra componenti	Irregolarità controllata dalla regola di connessione del codice.
Small-world	Reticolo locale con rewiring di Watts–Strogatz	Compromesso tra clustering e scorciatoie [2, 30].
Scale-free	Attaccamento preferenziale e nodi ad alto grado	Eterogeneità della distribuzione dei gradi [31, 32].

plementazione di ALC-PSO combina OmpSs e CUDA [21]. Sato et al. studiano la parallelizzazione multicore di MOPSO/D attraverso un virtual global best e una partizione dello spazio degli obiettivi [22]; altri lavori recenti riguardano PSO multicore-aware per edge offloading e varianti parallelizzate per il training di reti neurali [23, 24]. Questi contributi sono pertinenti al contesto HPC, ma non sono baseline numeriche dirette per l'ottimizzazione single-objective del presente protocollo: problema, dinamica, hardware e budget differiscono.

La distinzione qui proposta fra esecutori sincroni e algoritmi con dinamica modificata è complementare a tali lavori.

I modelli a isole sono un paradigma più generale degli algoritmi evolutivi: ciascuna isola evolve una popolazione locale e scambia individui secondo una topologia e una politica di migrazione. Vanneschi, Codecasa e Mauri confrontano metodi paralleli e distribuiti basati su co-evoluzione di sciami e multi-swarm a isole [25]. Da Silveira et al. analizzano topologie e politiche di migrazione in modelli a isole sincroni e asincroni [26], mentre lavori successivi considerano isole eterogenee e riconfigurabili [27]. Nel nostro modello le isole eseguono la stessa CPSO locale; la variabile sperimentale è la decomposizione della popolazione e la politica di migrazione.

2.3 Topologie sociali

La struttura del vicinato è una componente fondativa della PSO. Kennedy e Mendes documentano il ruolo della struttura di popolazione e delle topologie di informazione [28, 29]. Ring e global-best rappresentano riferimenti locali e globali; la griglia toroidale di von Neumann è un vicinato regolare bidimensionale. Le quattro famiglie aggiuntive ampliano lo spettro: wheel separa hub e periferie; il random connesso introduce irregolarità; Watts–Strogatz controlla il rewiring di un reticolo locale; Barabási–Albert introduce attaccamento preferenziale. Le ultime due richiamano modelli di rete indipendenti dalla PSO [30, 31]; topologie scale-free sono state studiate anche direttamente in algoritmi PSO [32].

Deng e Liu ricostruiscono reti di interazione emergenti dalla dinamica PSO e riportano proprietà small-world e distribuzioni dei gradi a coda pesante [2]. Il risultato motiva l'inclusione di grafi non regolari, ma non dimostra che imporre una famiglia small-world o scale-free migliori le prestazioni.

2.4 Baseline e testbed

Le dieci baseline sono selezionate per coprire meccanismi distinti: PSO global-best classica, inerzia lineare, fattore di costrizione, local-best ad anello, CLPSO, SL-PSO, SPSO-2011, FIPS, Competitive Swarm Optimizer e DMS-PSO. I riferimenti fondativi sono Kennedy ed Eberhart per la PSO originale [1], Shi ed Eberhart per l'inerzia [33], Shi ed Eberhart per lo studio empirico e la schedule lineare dell'inerzia [34], Clerc e Kennedy per il fattore di costrizione [35], CLPSO e SL-PSO per l'apprendimento sociale [36, 37], SPSO-2011 come riferimento standardizzato [38], FIPS per l'informazione fully-informed [39], CSO per l'apprendimento competitivo [40] e DMS-PSO per i sottosciami dinamici [41]. Le implementazioni Python includono adattamenti per uniformare popolazione, budget FE e gestione dei vincoli; i dettagli e le deviazioni rilevanti sono esplicitati nell'Appendice A.

I benchmark esterni comprendono COCO BBOB e BBOB-LargeScale e CEC 2013 LSGO [3–5, 42]. Le metriche anytime/ECDF e runtime-to-target sono preferite al solo rango del valore finale.

3 Modello matematico e semantica operativa

Indichiamo con N_p il numero di particelle, con D la dimensione dello spazio decisionale e con K il numero di sottointervalli temporali utilizzati nell'implementazione. Nelle sezioni sperimentali, N sarà talvolta usato come abbreviazione di N_p per mantenere la notazione delle tabelle di campagna.

La costruzione parte dalla PSO discreta. Per la particella i, posizione e velocità soddisfano

$$\boldsymbol{p}_i^{\lambda+1} = \boldsymbol{p}_i^{\lambda} + \boldsymbol{v}_i^{\lambda+1}, \tag{1}$$

$$\boldsymbol{v}_i^{\lambda+1} = \mu \boldsymbol{v}_i^{\lambda} + c_c r_1 \left(\boldsymbol{p}_i^{b} - \boldsymbol{p}_i^{\lambda}\right) + c_s r_2 \left(\boldsymbol{p}_{gb} - \boldsymbol{p}_i^{\lambda}\right), \tag{2}$$

qui μ è il peso d'inerzia, c_c e c_s sono i coefficienti cognitivo e sociale, $\boldsymbol{p}_i^b$ è la migliore posizione personale e $\boldsymbol{p}_{gb}$ è la migliore posizione globale. Le variabili casuali r_1, r_2 sono scalari e appartengono a $[0, 1]$ nella formulazione sorgente. L'articolo afferma che variano con l'indice di iterazione, ma non introduce per esse un indice di particella e non specifica esplicitamente se la stessa coppia sia condivisa dall'intero sciame oppure estratta indipendentemente per ogni particella. Questa distinzione è rilevante per l'implementazione, ma non è risolta dalla notazione dell'articolo.

Seguendo la costruzione a tempo continuo di [12], la posizione di una particella è rappresentata da una funzione vettoriale $\boldsymbol{p}_i(t) \in \mathbb{R}^D$, la cui velocità è $\dot{\boldsymbol{p}}_i(t)$. Si ottiene il seguente problema di Cauchy:

$$\begin{aligned} &\ddot{\boldsymbol{p}}_i(t) + (1-\mu(t))\,\dot{\boldsymbol{p}}_i(t) \\ &\qquad + \big(c_c r_1(t) + c_s r_2(t)\big)\,\boldsymbol{p}_i(t) = \boldsymbol{f}_i(t), \\ &\boldsymbol{p}_i(0) = \boldsymbol{p}_i^0, \qquad \dot{\boldsymbol{p}}_i(0) = \boldsymbol{v}_i^0. \end{aligned} \tag{3}$$

con termine forzante

$$\boldsymbol{f}_i(t) = c_c r_1(t)\,\boldsymbol{p}_i^b(t) + c_s r_2(t)\,\boldsymbol{p}_{gb}(t). \tag{4}$$

Introducendo la frequenza naturale non smorzata $\omega(t)$ e il rapporto di smorzamento $\zeta(t)$ si ottengono le definizioni utilizzate nell'articolo sorgente:

$$2\zeta(t)\omega(t) = 1-\mu(t), \qquad \omega^2(t) = c_c r_1(t) + c_s r_2(t). \tag{5}$$

Il modello continuo è quindi un oscillatore armonico smorzato. La legge del peso d'inerzia è lineare nell'indice di iterazione, da $\mu_{\max}$ all'inizio a $\mu_{\min}$ all'ultimo passo:

$$\mu(\lambda) = \mu_{\max} - \frac{\mu_{\max}-\mu_{\min}}{\lambda_{\max}}\lambda. \tag{6}$$

Nell'implementazione, per $K > 1$, l'indice di aggiornamento è $k = 0, \dots, K-1$ e la legge effettivamente utilizzata è

$$\mu_k = \mu_{\max} - \frac{\mu_{\max}-\mu_{\min}}{K-1}\,k. \tag{7}$$

Questa identificazione pone $\lambda_{\max} = K-1$; per $K \le 1$ il codice mantiene $\mu_k = \mu_{\max}$. È una convenzione d'indicizzazione dell'implementazione, coerente con una schedule che raggiunge $\mu_{\min}$ all'ultimo aggiornamento.

Problemi di Cauchy a coefficienti costanti a tratti. $\omega(t)$ e $\zeta(t)$ vengono approssimate come costanti a tratti su K sottointervalli di uguale ampiezza $\Delta t = T/K$:

$$\begin{aligned} &\omega(t) = \omega_k, \qquad \zeta(t) = \zeta_k, \\ &\qquad t \in [k\Delta t, (k+1)\Delta t], \qquad k = 0, \dots, K-1. \end{aligned} \tag{8}$$

In ciascun sottointervallo, lo stato e le posizioni best sono assunti al suo estremo sinistro. Il corrispondente problema di Cauchy a coefficienti costanti è

$$\begin{cases} \ddot{\boldsymbol{p}}_i(t) + 2\zeta_k\omega_k\,\dot{\boldsymbol{p}}_i(t) + \omega_k^2\,\boldsymbol{p}_i(t) = \boldsymbol{f}_{i,k}, \\ \boldsymbol{p}_i(k\Delta t) = \boldsymbol{p}_i^k, \qquad \dot{\boldsymbol{p}}_i(k\Delta t) = \boldsymbol{v}_i^k. \end{cases} \tag{9}$$

L'estremo della soluzione analitica fornisce $(\boldsymbol{p}_i^{k+1}, \boldsymbol{v}_i^{k+1})$ e diventa la condizione iniziale del sottointervallo successivo.

3.1 Corrispondenza tra il modello congelato e il nucleo C++

Nel caso di riferimento global-best, l'implementazione estrae per ogni particella i e intervallo k una coppia di valori scalari $r_{1,i}^k, r_{2,i}^k \in [0,1)$ e condivide ciascun valore fra tutte le D coordinate della particella. Si definiscano

$$\begin{aligned} a_i^k &= c_c r_{1,i}^k, & b_i^k &= c_s r_{2,i}^k, \\ q_i^k &= a_i^k + b_i^k = \omega_{i,k}^2, & \boldsymbol{F}_i^k &= a_i^k\,\boldsymbol{p}_i^{b,k} + b_i^k\,\boldsymbol{p}_{gb}^k, \\ \alpha_{i,k} &= \zeta_{i,k}\omega_{i,k} = \frac{1-\mu_k}{2}. \end{aligned} \tag{10}$$

Il codice risolve, coordinata per coordinata,

$$\ddot{x}_{i,j} + 2\alpha_{i,k}\dot{x}_{i,j} + q_i^k x_{i,j} = F_{i,j}^k, \tag{11}$$

Sotto l'interpretazione del termine forzante pesato dai numeri casuali, questa è algebricamente la stessa equazione dell'oscillatore congelato in (9). Quando la topologia configurata è global-best il codice utilizza il migliore globale; le configurazioni con leader di vicinato sostituiscono $\boldsymbol{p}_{gb}^k$ con un attrattore locale e costituiscono estensioni.

Estensioni operative: dominio limitato e topologie sociali Le sette topologie di comunicazione implementate sono: global-best; anello periodico di raggio uno; griglia toroidale di von Neumann con sé stessa e quattro vicini ortogonali; wheel; grafo random di tipo ER reso connesso mediante archi ponte; small-world di Watts–Strogatz con $k = 4$ e probabilità di riconnessione 0.15; Barabási–Albert con clique iniziale di dimensione $m+1$ e parametro di attaccamento $m = 2$. Soltanto global-best corrisponde alla regola sociale dell'articolo originale; le altre sei scelte sono estensioni. Queste famiglie sono motivate da modelli di rete consolidati e da studi sulle topologie PSO [28–32, 43].

Esecutori sincroni e invarianza della traiettoria In una generazione sincrona, tutti i leader sociali vengono selezionati dallo stesso snapshot pre-generazione $P^k = (\boldsymbol{p}_1^{b,k}, \dots, \boldsymbol{p}_{N_p}^{b,k})$. Ogni propagazione e valutazione della funzione obiettivo è assegnata a uno slot identificato dalla particella; gli aggiornamenti dei migliori personali e del global-best vengono poi applicati in ordine deterministico. Il backend master–slave parallelizza soltanto le valutazioni dell'obiettivo, mentre particle-parallel distribuisce anche l'inizializzazione e la propagazione locale delle particelle. I coefficienti di propagazione q_i^k e $\alpha_{i,k}$ sono comuni alle coordinate di una stessa particella.

Esecutore asincrono event-driven Dopo l'inizializzazione viene inserito in coda un evento $(i, 0)$ per ogni particella. Un evento (i,k) acquisisce sotto mutex lo snapshot del migliore personale locale e dell'attrattore sociale; rilascia il lock prima della propagazione $\Phi_{\Delta t}$ e della valutazione $J(x)$; acquisisce nuovamente il lock per il commit del miglioramento e dell'incumbent globale. Il successivo evento della stessa particella viene inserito soltanto dopo il completamento del precedente, quindi non esistono due eventi simultanei in volo per il medesimo indice. Lo stream casuale resta assegnato alla particella, ma non rende deterministico lo scheduling. Ogni particella esegue esattamente K eventi dopo la valutazione iniziale, per cui il budget rimane $N_{\mathrm{FE}} = N_p(K+1)$.

Modello a isole e migrazione La popolazione totale N_p viene suddivisa in M isole di uguale dimensione $n = N_p/M$, con $n \ge 2$. Ogni isola esegue localmente una generazione sincrona e usa una topologia intra-isola ammessa dal codice. Quando è previsto un evento di migrazione, il migliore personale di ciascuna isola viene copiato in uno snapshot immutabile; le sostituzioni sono applicate solo dopo aver costruito lo snapshot. Nella migrazione ad anello l'isola m invia il proprio incumbent all'isola $(m+1) \bmod M$; nella variante fully-connected ogni destinatario considera il migliore incumbent tra le altre isole. Il destinatario è la particella con il peggiore valore di

Tabella 2: Disegno della campagna architetturale interna.

Stadio	Disegno	Run	FE complessive
1. Topologie	6 funzioni, 2 dimensioni, 30 semi; serial-global, serial-ring, serial-von-Neumann	1 080	34 629 120
2. Asincronia	6 funzioni, 2 dimensioni, 30 semi; tre topologie, seriale e event-driven con 1/4 worker	3 240	103 887 360
3. Isole	6 funzioni, 2 dimensioni, 30 semi; 2/4/8 isole, topologie locali e politiche di migrazione	3 960	126 973 440
4. Strong scaling	`sphere_work`, $D = 30, 100$, $N = 64, 128$, $K = 80$, tre carichi, 10 semi; seriale, master–slave e particle-parallel	1 080	8 398 080
Totale		9 360	273 888 000

personal best e la sostituzione avviene soltanto se l'immigrato è strettamente migliore.

La sostituzione aggiorna congiuntamente la posizione corrente e il personal best del destinatario, assegna alla posizione corrente il valore dell'immigrato, azzera la velocità e conserva lo stream pseudocasuale del destinatario. La migrazione non richiede nuove FE perché il valore dell'immigrato è già disponibile. Il codice non migra dopo l'ultima generazione, richiede che N_p sia divisibile per M, che ogni isola abbia almeno due particelle e che il numero di worker non superi il numero di isole. L'intervallo, la topologia di migrazione e la politica di sostituzione sono parametri di configurazione e vengono riportati nell'Appendice C.

Tutte le modalità considerate mantengono il budget

$$N_{\mathrm{FE}} = N_p(K + 1), \tag{12}$$

comprensivo della valutazione iniziale di ogni particella.

4 Protocollo sperimentale

4.1 Campagna architetturale interna

La campagna architetturale predefinita è articolata in quattro stadi. Gli stadi 1–3 studiano qualità della ricerca e cambiamenti della dinamica; lo stadio 4 misura esclusivamente il tempo dell'esecutore su un carico sintetico. Per gli stadi di qualità sono considerate Sphere, Rastrigin, Rosenbrock, Ackley, Griewank e Salomon, con $D \in \{30, 100\}$, popolazione totale $N = 64$, $K = 500$ aggiornamenti e 30 semi appaiati. Ogni run usa $N(K+1) = 32\,064$ FE. I parametri CPSO sono quelli specificati nella Sezione 3; in particolare, le configurazioni correnti fissano $\Delta t = 0.35$.

Il primo stadio isola l'effetto della topologia in esecuzione seriale. Il secondo valuta l'aggiornamento event-driven a uno e quattro worker, a parità di FE e popolazione totale. Il terzo studia la decomposizione in 2, 4 e 8 isole, nonché intervalli e topologie di migrazione.

Espansione a sette topologie La campagna principale usa global-best, ring e von Neumann per limitare il numero di combinazioni con le semantiche di esecuzione. Una campagna complementare estende il confronto a tutte e sette le famiglie, aggiungendo wheel, random connessa, small-world e scale-free. La configurazione prevede un confronto di qualità su sei funzioni, $D \in \{30, 100\}$, $N = 64$, $K = 200$ e 30 semi, oltre a uno stadio di costo su `sphere_work` con $D = 30$, $N = 64$, $K = 150$ e 30 semi. Il disegno completo comprende 3 150 run.

I parametri dei grafi sono random mean-degree target 4, Watts–Strogatz $k = 4, \beta = 0.15$ e Barabási–Albert $m = 2$; ciascun grafo è statico per run.

Misura dello scaling sincrono Lo stadio 4 usa `sphere_work`, una funzione sintetica basata su Sphere con costo aritmetico controllabile. Sono considerati $D \in \{30, 100\}$, $N \in \{64, 128\}$, $K = 80$, carichi di 0, 512 e 4 096 iterazioni per valutazione e 10 semi. Per ciascun backend sincrono si misurano 1, 2, 4 e 8 worker.

Lo speedup e l'efficienza sono

$$S(p) = \frac{T_{\mathrm{serial}}}{T_p}, \qquad E(p) = \frac{S(p)}{p}. \tag{13}$$

Asincronia e isole non sono interpretate come speedup del medesimo algoritmo. Per l'interpretazione si registrano separatamente tempi di propagazione, valutazione, sincronizzazione e overhead dell'esecutore; il costo di propagazione può essere riutilizzato per coordinata e non è assunto proporzionale a D.

Unità statistiche e inferenza Nella campagna interna, i seed sono repliche appaiate all'interno di ciascuna cella funzione-dimensione, le sei funzioni costituiscono le unità di livello superiore.

Questa scelta evita pseudoreplicazione, ma impone un limite matematico alla potenza del test signed-rank esatto bilaterale. Con sei unità indipendenti, il più piccolo valore p teoricamente raggiungibile senza pareggi è

$$p_{\min} = \frac{2}{2^6} = 0.03125. \tag{14}$$

Con correzione di Holm per almeno due confronti, anche il minimo aggiustato è almeno 0.0625. Di conseguenza, la campagna interna a sei funzioni non può produrre un rifiuto confermativo al livello 0.05 per una famiglia di almeno due confronti basata su tale test. Le sue analisi statistiche sono quindi diagnostiche/esplorative: verranno riportati effetti, intervalli, distribuzioni per funzione-dimensione e risultati descrittivi.

La suite BBOB offre 24 funzioni e CEC LSGO 15 funzioni; le istanze COCO e le ripetizioni non saranno conteggiate come funzioni indipendenti. Le correzioni per confronti multipli saranno definite a priori. Le metriche di successo useranno target graduati ove supportati e curve anytime/ECDF; l'ERT sarà censurato al budget per le run senza successo e accompagnato dal conteggio dei successi. Il target interno 10^{-8} sarà mantenuto soltanto per le funzioni con ottimo nullo e non sarà considerato una misura sufficiente da solo.

Benchmark esterni e budget La valutazione esterna usa COCO `bbob`, COCO `bbob-largescale` e CEC 2013 LSGO [3–5, 42].

Il budget $10^4 D$ per BBOB e BBOB-LargeScale è una scelta del progetto. Per CEC 2013 LSGO sono previsti 25 seed e

Tabella 3: Protocollo esterno definito nelle configurazioni correnti.

Suite	Problemi	Dimensioni	Istanze/semi	Budget massimo per run
COCO BBOB	24 funzioni	2, 3, 5, 10, 20, 40	15 istanze	$10^4 D$ FE
COCO BBOB-LargeScale	24 funzioni	20, 40, 80, 160, 320, 640	15 istanze	$10^4 D$ FE
CEC 2013 LSGO	15 funzioni	1000 variabili nominali; F13 e F14 hanno 905 variabili effettive	25 seed	$3 \cdot 10^6$ FE

checkpoint a $1.2 \cdot 10^5$, $6 \cdot 10^5$ e $3 \cdot 10^6$ FE. Le funzioni F13 e F14 hanno 905 variabili effettive.

Con 11 metodi, le configurazioni complete prevedono 23 760 run per ciascuna suite COCO e 4 125 run per CEC. Il carico pianificato è di 3.168 miliardi di FE per BBOB, 49.896 miliardi per BBOB-LargeScale e 12.375 miliardi per CEC, pari a 65.439 miliardi di FE complessive. Questo carico è sostanziale, soprattutto perché le callback esterne sono seriali.

Baseline discrete e comparabilità Il confronto esterno include la CPSO e dieci baseline discrete: classic global-best, inertia global-best, constriction global-best, inertia local-best ring, CLPSO, SL-PSO, SPSO-2011, FIPS ring, CSO e DMS-PSO. Il riepilogo dei meccanismi e le formule usate sono riportati nell'Appendice A.

La popolazione esterna è fissata a $N = 40$ per uniformare il budget FE. La configurazione è intesa come confronto a popolazione controllata, la politica di clipping e restituzione è comune dove implementabile; SPSO-2011 mantiene la propria regola di confinamento documentata nell'Appendice. Le impostazioni di velocità massima sono riportate esplicitamente: i metodi classici con inerzia adottano un limite pari a 0.2 della larghezza del dominio; la variante con fattore di costrizione, FIPS, SPSO-2011, SL-PSO e CSO non adottano tale limite nella configurazione corrente; CLPSO e DMS-PSO lo adottano.

La formula di apprendimento SL-PSO è stata verificata rispetto alla fonte primaria di Cheng e Jin: il fattore sociale $\gamma = \beta D/M$ e la probabilità originale $P_i^L = (1-(i-1)/m)^{\alpha \log\lceil D/M \rceil}$, con rango one-based $i = 1, \ldots, m-1$, coincidono con il codice espresso nell'indice zero-based $r = 0, \ldots, N-2$, $P_r = (1 - r/N)^{\alpha \log\lceil D/M \rceil}$. La principale adattazione rispetto al protocollo originale è la popolazione fissata a $N = 40$ per rendere uguale il budget FE, invece della regola raccomandata $m = M + \lfloor D/10 \rfloor$. SL-PSO viene quindi riportata come implementazione con popolazione fissata.

5 Risultati sperimentali

Questa sezione è predisposta per l'inserimento dei risultati delle campagne congelate. Alla data della presente versione non vengono presentati valori confirmatory: le celle contrassegnate con `TBD` sono segnaposto intenzionali, non risultati stimati. I pilot locali possono essere visualizzati esclusivamente in figure identificate come esplorative, ma non vengono usati per compilare le tabelle confirmatory. Ogni tabella definitiva dovrà riportare il numero di run completate, le FE effettive, la copertura dei problemi e le esclusioni motivate.

5.1 Equivalenza delle modalità sincrone (RQ1)

Il controllo confronta l'intero record osservato rispetto al riferimento seriale: valore e posizione migliori, storia best-so-far, checkpoint FE e numero di valutazioni. L'audit implementato registra il numero di coppie verificate e attese mediante uguaglianza esatta dei campi; non registra una norma di errore numerica massima. La tabella riassume il disegno del pilot M2 (sei funzioni, due dimensioni e cinque seed: 60 coppie per backend e numero di worker).

5.2 Strong scaling e costo dell'esecutore (RQ2)

Lo scaling è riportato esclusivamente per configurazioni che superano il controllo di equivalenza. Il pilot M2 usa la funzione sintetica `sphere_work`, popolazione $N = 64$, dimensioni $D \in \{30, 100\}$, due carichi (512 e 4096 iterazioni di lavoro per valutazione) e worker $p \in \{1, 2, 4\}$. La tabella riassume il caso a quattro worker; la figura mostra l'intera curva per $p = 1, 2, 4$. Le misure sono locali alla macchina M2 e non costituiscono una caratterizzazione HPC generalizzabile.

La decomposizione temporale misurata (inizializzazione, setup topologico, propagazione, selezione sociale, valutazione e residuo) è riportata nell'Appendice D, Tabella D.17 e Figura D.9. Il residuo non viene interpretato come una misura isolata della sincronizzazione: comprende tutte le attività non strumentate.

5.3 Aggiornamento asincrono e modello a isole (RQ3)

L'asincronia e il modello a isole vengono confrontati con riferimenti seriali a popolazione $N = 64$ e budget $N(K + 1) = 12\,864$ FE/run ($K = 200$). Le differenze sono interpretate come effetti di dinamiche differenti, non come speedup. Il pilot asincrono considera topologie global e ring con 2 e 4 worker; il pilot a isole considera due o quattro sottopopolazioni, migrazione assente oppure migrazione ring/fully-connected ogni 20 iterazioni. Le configurazioni effettivamente eseguite sono riportate nella tabella; non sono inclusi livelli di worker, topologie o intervalli non simulati.

5.4 Topologie sociali (RQ4)

Le sette topologie sono confrontate sulla stessa popolazione ($N = 64$), sulle sei funzioni native, su $D = 30, 100$ e su cinque seed. La tabella 8 riporta le specifiche dichiarate nella configurazione, la mediana degli effetti appaiati rispetto al global-best e il rango medio (rango 1 = migliore; i pareggi ricevono rango medio). Le statistiche effettive dei grafi (grado,

Tabella 4: Audit esplorativo di equivalenza per le modalità sincrone. Ogni riga del pilot contiene 60 confronti appaiati (sei funzioni, due dimensioni, cinque seed). "Identiche" indica l'uguaglianza esatta dei campi di traiettoria registrati; i mismatch sono calcolati come coppie attese meno coppie verificate.

Backend	Worker	Coppie attese	Coppie identiche	Mismatch	Identità (%)
Master–slave	1	60	TBD	TBD	TBD
Master–slave	2	60	TBD	TBD	TBD
Master–slave	4	60	TBD	TBD	TBD
Particle-parallel	1	60	TBD	TBD	TBD
Particle-parallel	2	60	TBD	TBD	TBD
Particle-parallel	4	60	TBD	TBD	TBD

Tabella 5: Template di riepilogo dello strong scaling al punto $p = 4$. Il pilot usa $N = 64$, $K = 80$ e quindi 5 184 FE/run; i due carichi sintetici sono 512 e 4096 iterazioni per valutazione. La Figura 2 mostra anche $p = 1, 2, 4$.

D	N	Lavoro	FE/run	Backend	T_4 (s)	$S(4)$	IC 95% $S(4)$	$E(4)$
30	64	512	5 184	Master–slave	TBD	TBD	TBD	TBD
30	64	512	5 184	Particle-parallel	TBD	TBD	TBD	TBD
30	64	4096	5 184	Master–slave	TBD	TBD	TBD	TBD
30	64	4096	5 184	Particle-parallel	TBD	TBD	TBD	TBD
100	64	512	5 184	Master–slave	TBD	TBD	TBD	TBD
100	64	512	5 184	Particle-parallel	TBD	TBD	TBD	TBD
100	64	4096	5 184	Master–slave	TBD	TBD	TBD	TBD
100	64	4096	5 184	Particle-parallel	TBD	TBD	TBD	TBD

clustering, diametro e cammino medio) non sono presenti nei record del pilot e non vengono ricostruite dalle sole etichette topologiche; la loro esportazione richiederà una modifica strumentale separata.

5.5 Benchmark esterni: COCO BBOB e BBOB-LargeScale (RQ5)

Le ECDF e le curve runtime-to-target COCO sono le figure confermative previste. Il pilot BBOB M2 attualmente disponibile copre sette funzioni (F1, F5, F8, F13, F17, F21, F24), dimensioni 5 e 20, istanze 1–5 e undici metodi. I record JSONL permettono analisi descrittive per istanza; non sostituiscono gli output Observer richiesti per ECDF/ERT. L'ECDF verrà inserita solo dopo aver verificato la copertura dei file `.info` per tutti i metodi. BBOB-LargeScale resta separato e non dispone ancora di risultati.

5.6 Benchmark esterno: CEC 2013 LSGO

Per CEC 2013 LSGO si riportano i valori best-so-far ai tre checkpoint FE previsti, le distribuzioni per funzione e il riepilogo dei successi. Le funzioni F13 e F14 hanno dimensione effettiva pari a 905. La campagna non è ancora presente nei risultati disponibili e il driver deve essere verificato per accettare esplicitamente queste due dimensioni prima dell'esecuzione.

5.7 Riepilogo delle domande di ricerca

La tabella 11 sarà compilata dopo le campagne. Ogni risposta dovrà distinguere una proprietà matematica dimostrata, un controllo implementativo superato, un risultato empirico e un'ipotesi non verificata.

I risultati estesi per funzione, dimensione, seed, topologia e baseline saranno collocati nell'Appendice D, così da mantenere il corpo principale concentrato sulle risposte alle RQ. Tutte le figure ottenute dai pilot M2 sono contrassegnate come esplorative; i valori delle tabelle restano TBD fino al congelamento e all'esecuzione delle campagne confermative.

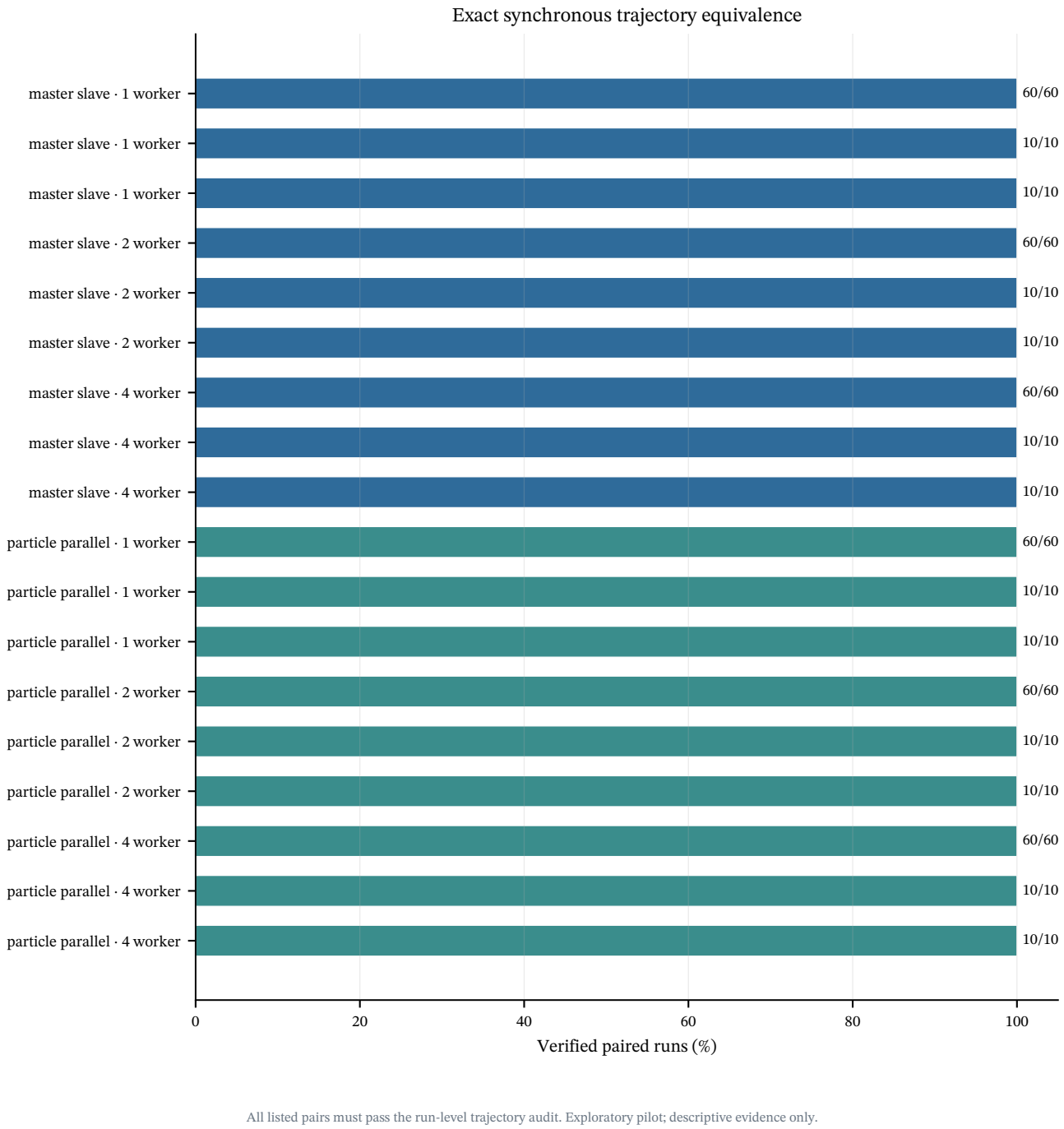


Figura 1: Equivalenza delle traiettorie e degli incumbent tra il riferimento seriale e gli esecutori sincroni. Risultati del pilot M2, riportati a fini esplorativi.

Tabella 6: Template per la qualità dell'aggiornamento asincrono nel pilot M2. Ogni riga aggrega 30 run (sei funzioni per cinque seed) a dimensione fissata; la mediana e l'IQR sono ottenuti prima sui seed entro funzione e poi tra le sei funzioni. Il target di successo è 10^{-8}; FE/run = 12 864.

Metodo	D	Worker	Topologia	Mediana tra funzioni di $\log_{10}(f_{\text{best}})$ [IQR]	Successi/30	ERT (FE)
Event-driven	30	2	Global	TBD	TBD	TBD
Event-driven	30	4	Global	TBD	TBD	TBD
Event-driven	30	2	Ring	TBD	TBD	TBD
Event-driven	30	4	Ring	TBD	TBD	TBD
Event-driven	100	2	Global	TBD	TBD	TBD
Event-driven	100	4	Global	TBD	TBD	TBD
Event-driven	100	2	Ring	TBD	TBD	TBD
Event-driven	100	4	Ring	TBD	TBD	TBD

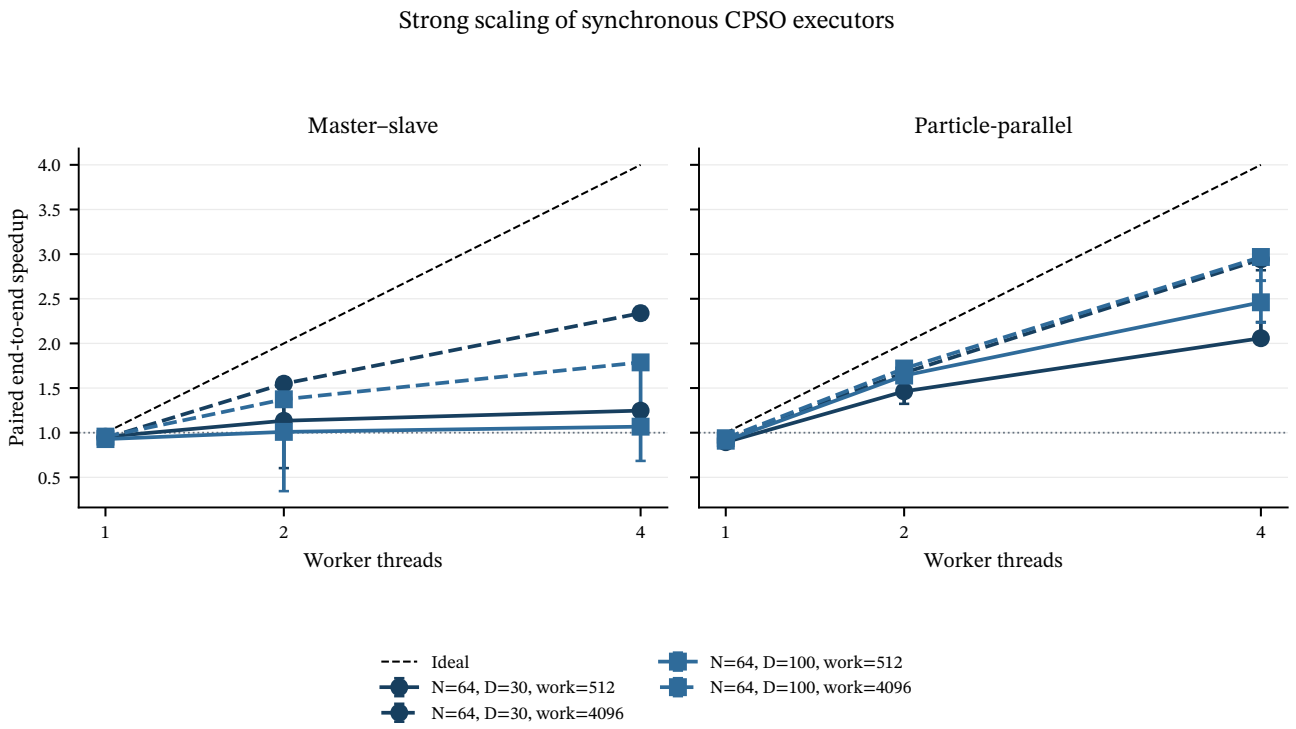


Figura 2: Strong scaling dei backend sincroni su `sphere_work` per $D = 30, 100$, due carichi sintetici e $p = 1, 2, 4$ worker. Tempi del pilot esplorativo misurati sul MacBook Air M2.

Tabella 7: Template per le quattro configurazioni a isole effettivamente eseguite nel pilot M2. Ogni configurazione è stata simulata per $D = 30$ e $D = 100$, sei funzioni e cinque seed; FE/run = 12 864. Per quattro isole e intervallo 20 si hanno nove epoche di migrazione e 36 tentativi per run (uno per isola e per epoca).

D	Isole	Pop./isola	Worker	Topologia intra	Migrazione	Intervallo	Accettate (%)	Successi/30	FE/run
30	2	32	2	Global	Nessuna	–	–	TBD	12 864
100	2	32	2	Global	Nessuna	–	–	TBD	12 864
30	4	16	4	Global	Ring	20	TBD	TBD	12 864
100	4	16	4	Global	Ring	20	TBD	TBD	12 864
30	4	16	4	Global	Fully connected	20	TBD	TBD	12 864
100	4	16	4	Global	Fully connected	20	TBD	TBD	12 864
30	4	16	4	Ring	Ring	20	TBD	TBD	12 864
100	4	16	4	Ring	Ring	20	TBD	TBD	12 864

Tabella 8: Topologie e parametri di generazione dichiarati nella configurazione del pilot M2. Le statistiche misurate del grafo non sono contenute nei record e restano da strumentare.

Topologia	Specifica dichiarata	Mediana $\Delta \log_{10}(f_{\text{best}})$ vs global	Rango medio
Global-best	Informazione globale condivisa	0 (riferimento)	TBD
Ring	Anello periodico a due vicini	TBD	TBD
Von Neumann	Griglia toroidale 8×8	TBD	TBD
Wheel	Grafo wheel su 64 particelle	TBD	TBD
Random	Grafo random, parametro di grado 4	TBD	TBD
Watts–Strogatz	Grado configurato 4, probabilità di rewiring 0.15	TBD	TBD
Barabási–Albert	Parametro di grado configurato 4	TBD	TBD

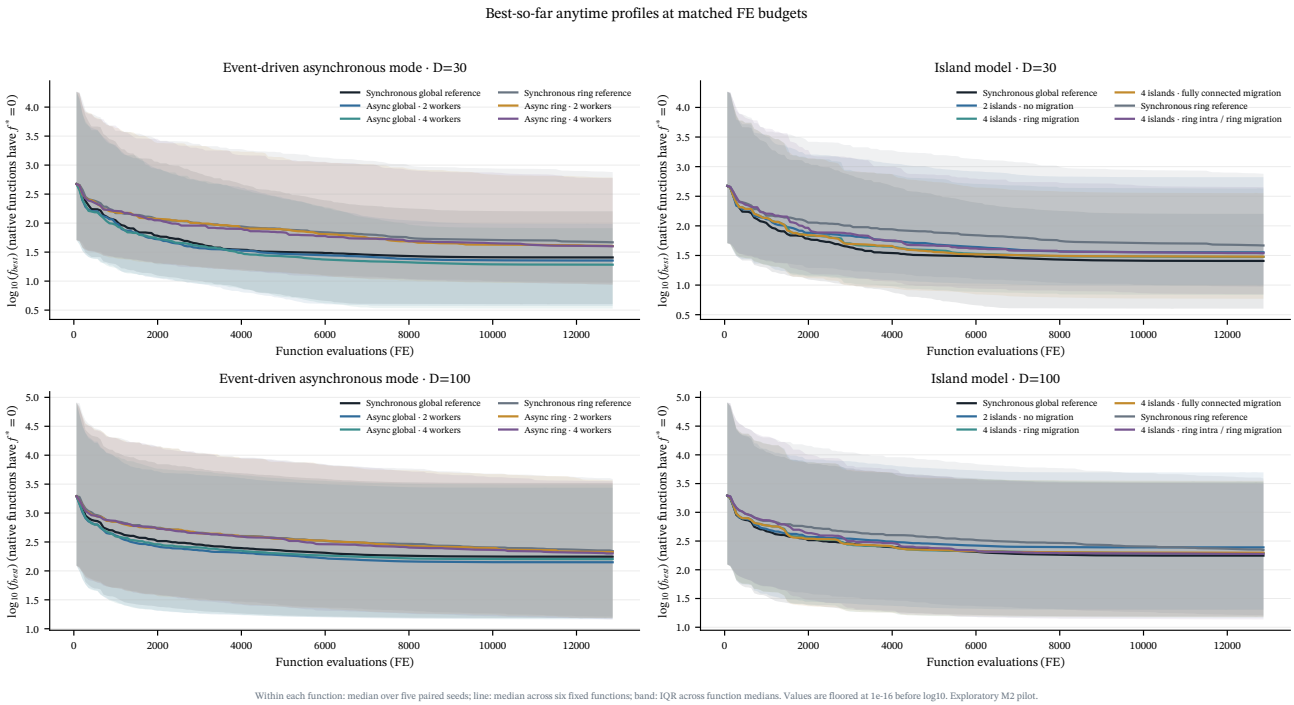


Figura 3: Confronto anytime tra riferimento sincrono, aggiornamento event-driven e modelli a isole a budget FE abbinato. Curve ricavate dal pilot esplorativo M2; bande e linee sono aggregazioni descrittive.

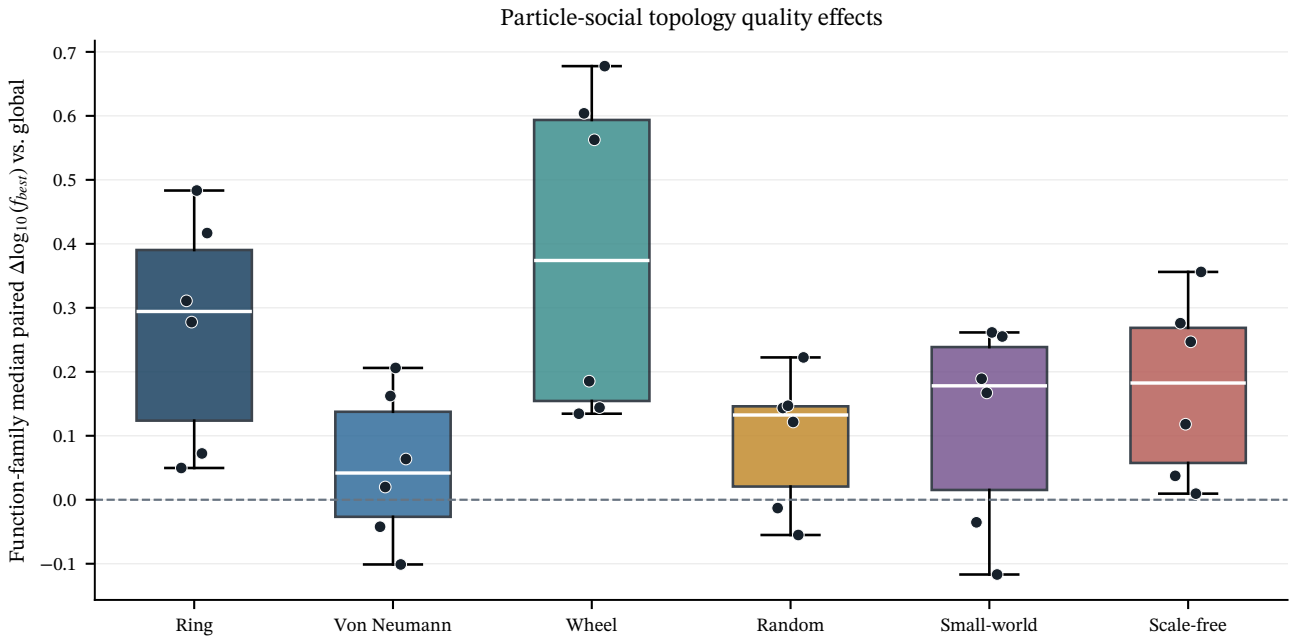


Figura 4: Effetti appaiati di qualità per le sette famiglie topologiche nella campagna dedicata. I punti rappresentano le sei funzioni; valori negativi favoriscono la topologia rispetto al global-best. Pilot M2 esplorativo.

Tabella 9: Template per il riepilogo COCO. Compilare separatamente per BBOB e BBOB-LargeScale; riportare ECDF/ERT per più target, non soltanto il valore finale.

Metodo	Funzioni coperte	Run complete	Successi	ERT target 1	ERT target 2	ECDF/RTT riferimento
CPSO	TBD	TBD	TBD	TBD	TBD	TBD
Classic gbest	TBD	TBD	TBD	TBD	TBD	TBD
Inertia gbest	TBD	TBD	TBD	TBD	TBD	TBD
Constriction gbest	TBD	TBD	TBD	TBD	TBD	TBD
Inertia lbest ring	TBD	TBD	TBD	TBD	TBD	TBD
CLPSO	TBD	TBD	TBD	TBD	TBD	TBD
SL-PSO (configurazione implementata)	TBD	TBD	TBD	TBD	TBD	TBD
SPSO-2011	TBD	TBD	TBD	TBD	TBD	TBD
FIPS ring	TBD	TBD	TBD	TBD	TBD	TBD
Competitive Swarm Optimizer	TBD	TBD	TBD	TBD	TBD	TBD
DMS-PSO	TBD	TBD	TBD	TBD	TBD	TBD

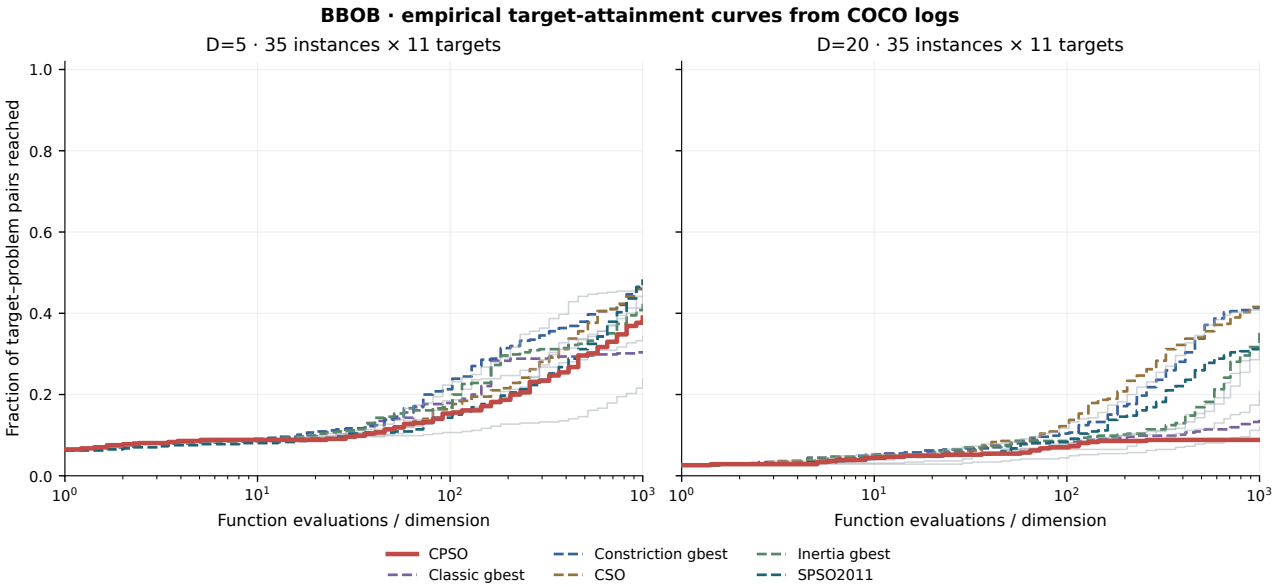


Figura 5: ECDF del benchmark COCO BBOB ottenute con il post-processing ufficiale.

Figura da generare
`figures/results/fig_coco_bbob_ls_ecdf.pdf`
ECDF COCO BBOB-LargeScale.

Figura 6: ECDF del benchmark COCO BBOB-LargeScale.

Tabella 10: Template per CEC 2013 LSGO ai checkpoint FE predefiniti. Ogni cella riporta mediana [IQR] tra i 25 seed della funzione; l'aggregazione tra funzioni è separata.

Metodo	Run complete	Successi	$1.2 \cdot 10^5$ FE	$6 \cdot 10^5$ FE	$3 \cdot 10^6$ FE	Rango medio
CPSO	TBD	TBD	TBD	TBD	TBD	TBD
Classic gbest	TBD	TBD	TBD	TBD	TBD	TBD
Inertia gbest	TBD	TBD	TBD	TBD	TBD	TBD
Constriction gbest	TBD	TBD	TBD	TBD	TBD	TBD
Inertia lbest ring	TBD	TBD	TBD	TBD	TBD	TBD
CLPSO	TBD	TBD	TBD	TBD	TBD	TBD
SL-PSO (configurazione implementata)	TBD	TBD	TBD	TBD	TBD	TBD
SPSO-2011	TBD	TBD	TBD	TBD	TBD	TBD
FIPS ring	TBD	TBD	TBD	TBD	TBD	TBD
Competitive Swarm Optimizer	TBD	TBD	TBD	TBD	TBD	TBD
DMS-PSO	TBD	TBD	TBD	TBD	TBD	TBD

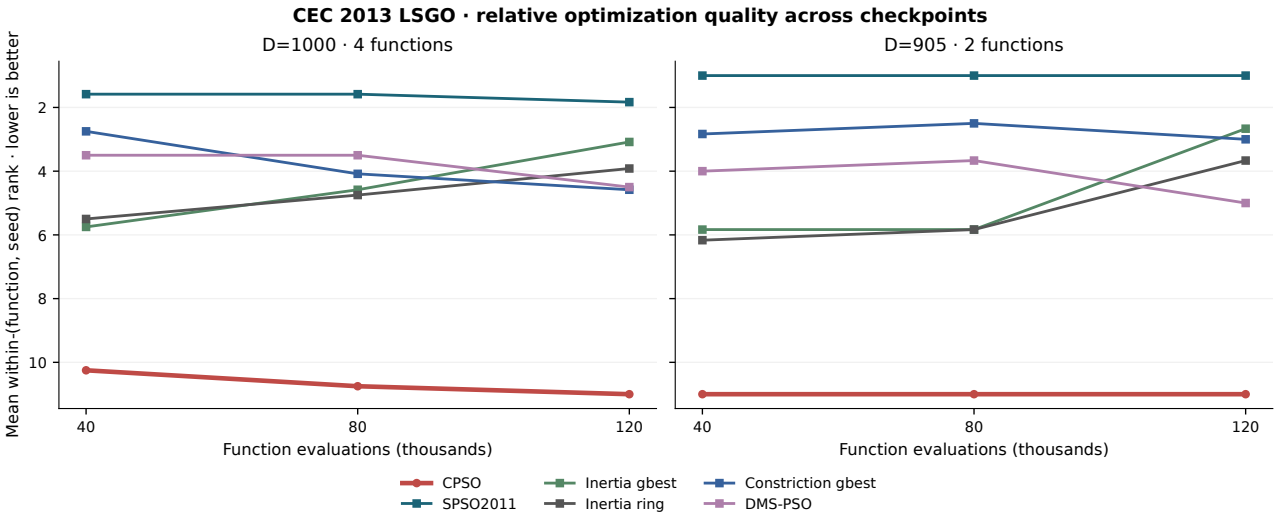


Figura 7: Confronto CEC 2013 LSGO ai checkpoint FE del protocollo.

Tabella 11: Template di sintesi delle RQ. Le conclusioni non devono eccedere la copertura effettivamente completata.

Domanda	Evidenza quantitativa da riportare	Risposta supportata dai dati
RQ1	Identità delle traiettorie, mismatch e condizioni di equivalenza	TBD
RQ2	Speedup, efficienza, costo per componente e intervalli	TBD
RQ3	Qualità anytime e successi a FE matched per asincronia/isole	TBD
RQ4	Effetti appaiati di qualità e specifiche di generazione delle sette topologie; descrittori misurati dei grafi ancora da strumentare	TBD
RQ5	ECDF/ERT COCO e checkpoint CEC per le undici implementazioni	TBD

6 Discussione

La discussione definitiva dipenderà dai risultati confermativi e non viene anticipata in questa versione. La sua struttura separa deliberatamente proprietà del modello, correttezza dell'implementazione, qualità della ricerca e prestazioni dell'esecutore: nessuno di questi livelli può sostituire gli altri.

6.1 Semantica ed equivalenza degli esecutori

L'interpretazione di RQ1 dovrà riportare se l'identità osservata sia stata verificata sull'intera cronologia e per quali combinazioni di worker e topologia. Eventuali mismatch dovranno essere classificati (RNG, ordine di commit, floating-point, funzione obiettivo o gestione dello stato) prima di utilizzare i tempi per uno speedup.

6.2 Granularità del lavoro e scaling

La risposta a RQ2 dovrà essere interpretata alla luce del rapporto fra costo della propagazione analitica, valutazione obiettivo e sincronizzazione. Lo strong scaling su `sphere_work` è un test controllato dell'esecutore, non una simulazione fisica costosa. Le misure su hardware fanless non consentono inferenze generali su nodi HPC; un eventuale caso d'uso con valutazione costosa ed eterogenea sarebbe un'estensione distinta. L'asincronia non viene presentata come intrinsecamente migliore o peggiore: la qualità su costo uniforme e il load balancing con costo eterogeneo sono due domande diverse.

6.3 Casualità scalare e validità algoritmica

La casualità scalare condivisa tra coordinate è mantenuta per fedeltà alla CPSO di Orlando–Ricciardello. La limitazione di sottospazio descritta nell'Appendice B vale in assenza di interventi della mappa non lineare dei vincoli; non dimostra da sola l'irraggiungibilità dell'ottimo. La sua possibile rilevanza in alta dimensione deve essere valutata con indicatori di rango, frequenza di attivazione dei vincoli e benchmark con ottimi traslati/ruotati. Una variante a casualità per coordinata, se introdotta, sarà trattata esclusivamente come ablation separata e non come CPSO di riferimento.

6.4 Topologie, isole e migrazione

I risultati topologici saranno letti insieme a grado, clustering, diametro e connettività. In assenza di un disegno degree-matched, non si attribuiranno le differenze alla sola geometria del grafo. Nel modello a isole la dimensione delle sottopopolazioni e la politica di migrazione cambiano simultaneamente l'organizzazione informativa; i risultati dovranno quindi distinguere l'effetto della decomposizione da quello della migrazione e riconoscere il limite delle isole piccole.

6.5 Posizionamento e minacce alla validità

Il confronto con le dieci baseline consente conclusioni soltanto entro la famiglia PSO e con le configurazioni dichiarate. Non implica superiorità rispetto a CMA-ES, differential evolution o a tutte le metaeuristiche di ottimizzazione continua. Le principali minacce da discutere saranno: sei sole funzioni interne e insufficiente potenza per i test pairwise Holm; center bias dell'insieme interno; casualità scalare della CPSO; popolazione esterna fissa $N = 40$; differenze tra generatori casuali e inizializzazioni; adattamenti delle baseline; callback Python seriali; budget esterno di 65.439 miliardi di FE; hardware locale e variabilità termica; non determinismo di scheduling asincrono. L'Appendice C e i manifest delle run dovranno documentare ogni deviazione dal protocollo congelato.

7 Conclusioni

Le conclusioni saranno compilate soltanto dopo il completamento e l'audit delle campagne confermative. La versione definitiva risponderà alle cinque domande di ricerca separando: (i) la correttezza del contratto sincrono e l'equivalenza della

traiettoria; (ii) lo scaling misurato sulla piattaforma effettivamente descritta; (iii) gli effetti sulla qualità dell'aggiornamento asincrono e delle isole; (iv) il confronto tra le sette famiglie topologiche; (v) il posizionamento della CPSO di Orlando–Ricciardello rispetto alle dieci baseline PSO sui benchmark esterni.

Saranno riportate soltanto conclusioni sostenute dai dati, con la copertura dei problemi e i limiti statistici esplicitati. In particolare, i risultati della campagna interna a sei funzioni saranno trattati come evidenza diagnostica; la terminologia CPSO resterà riservata alla formulazione di Orlando–Ricciardello e qualsiasi variante della casualità sarà presentata separatamente. Non saranno formulate affermazioni di superiorità universale o di scaling HPC oltre il protocollo realmente misurato.

Disponibilità di dati e codice

Il codice sorgente e le configurazioni sperimentali sono inclusi nella repository di sviluppo associata al manoscritto. Le configurazioni confirmatory sono definite nella repository; alla data di questa versione non sono ancora state eseguite integralmente. I pilot locali saranno etichettati come esplorativi e non sostituiranno i risultati confirmatory. I dati grezzi, i manifest e le figure verranno associati alla revisione del manoscritto che li discuterà.

A Formulazioni e configurazioni delle baseline PSO

Le baseline sono algoritmi PSO discreti in spazio decisionale continuo e sono implementate in moduli NumPy distinti dal nucleo CPSO C++. In questa appendice si documentano le formule e le scelte effettivamente presenti nel codice. Le adattazioni per budget FE comune non vengono presentate come parte delle formulazioni originali.

A.1 Parametri e adattamenti comuni

La popolazione esterna è fissata a $N = 40$ per tutti gli undici metodi e la valutazione è limitata da un budget FE identico per problema e algoritmo. Il seed deriva da un hash stabile dell'identificatore del problema e del seme di campagna; generatori diversi e implementazioni indipendenti implicano che il pairing del seed non garantisce inizializzazione coordinata identica. La velocità iniziale è in generale campionata in $[-0.1w_j, 0.1w_j]$, dove $w_j = u_j - \ell_j$; SL-PSO e Competitive Swarm Optimizer iniziano con velocità nulla, mentre SPSO-2011 usa la propria inizializzazione nel box normalizzato.

Tabella A.12: Configurazioni effettive delle baseline nel codice Python. $V_{\max}$ è espresso come frazione della larghezza del dominio.

Metodo	Parametri/controlli principali	$V_{\max}$	v^0	Nota di adattamento
Classic global-best	$c_1 = c_2 = 2$, $w = 1$	$0.2w_j$	$U[-0.1w_j, 0.1w_j]$	PSO gbest senza inerzia variabile; clipping e restituzione comuni.
Inertia global-best	$c_1 = c_2 = 2.05$, $w : 0.9 \to 0.4$	$0.2w_j$	$U[-0.1w_j, 0.1w_j]$	Schedule lineare associata allo studio empirico di Shi–Eberhart 1999.
Constriction global-best	$c_1 = c_2 = 2.05$, χ di Clerc–Kennedy	Nessuno	$U[-0.1w_j, 0.1w_j]$	Nessun limite di velocità nella configurazione corrente.
Inertia local-best ring	$c_1 = c_2 = 2.05$, $w : 0.9 \to 0.4$, raggio 1	$0.2w_j$	$U[-0.1w_j, 0.1w_j]$	Snapshot sincrono con vicinato periodico.
CLPSO	$c = 1.49445$, $w : 0.9 \to 0.4$, refresh 7	$0.2w_j$	$U[-0.1w_j, 0.1w_j]$	Esemplari per coordinata e probabilità di apprendimento dipendente dall'indice.
SL-PSO	$M = 100$, $\alpha = 0.5$, $\beta = 0.01$	Nessuno	0	Formula di apprendimento conforme; popolazione fissata a 40 per parità FE.
SPSO-2011	$K_{\text{informant}} = 3$, $c = 0.5 + \ln 2$, $\omega = (2 \ln 2)^{-1}$	Nessuno	Box normalizzato	Direzioni uniformi e raggio uniforme; regola di confinamento SPSO-2011.
FIPS ring	$\phi = 4.1$, due vicini, χ di costrizione	Nessuno	$U[-0.1w_j, 0.1w_j]$	Coefficienti casuali per coordinata, ring escluso il self.
Competitive Swarm Optimizer	$\varphi = 0.1$, matching casuale	Nessuno	0	Soli perdenti aggiornati; $N/2$ FE per generazione se N è pari.
DMS-PSO	$c_1 = c_2 = 2$, $w : 0.9 \to 0.2$, gruppo obiettivo 3, regroup 5	$0.2w_j$	$U[-0.1w_j, 0.1w_j]$	Variante base senza ricerca locale quasi-Newton.

Per i metodi con limite di velocità, $V_{\max} = 0.2(u_j - \ell_j)$ per coordinata. Le baseline che non lo usano non ricevono un limite implicito. La mappa clamp–restitution è comune per i metodi che non prescrivono una regola di confinamento propria; SPSO-2011 mantiene il proprio clipping e inversione della componente uscente. Le iterazioni sono sincrone rispetto a snapshot congelati; l'ordine delle valutazioni non crea un aggiornamento asincrono accidentale.

A.2 Fattore di costrizione

La PSO con fattore di costrizione usa

$$\chi = \frac{2}{\left|2 - \varphi - \sqrt{\varphi^2 - 4\varphi}\right|} = \frac{2}{\varphi - 2 + \sqrt{\varphi^2 - 4\varphi}}, \quad \varphi = c_1 + c_2 > 4. \tag{A.15}$$

Per $c_1 = c_2 = 2.05$, $\varphi = 4.1$ e $\chi \simeq 0.7298$. Il sorgente Python usa la seconda forma, corretta; il denominatore alternativo $4 - \varphi + \sqrt{\varphi^2 - 4\varphi}$ comparso in una versione del manoscritto era un refuso redazionale e non corrisponde all'implementazione.

A.3 Inerzia e CLPSO

La baseline con inerzia utilizza w_t decrescente linearmente da 0.9 a 0.4, con coefficienti $c_1 = c_2 = 2.05$. L'introduzione del peso d'inerzia è associata a Shi ed Eberhart 1998, mentre lo studio empirico da cui si riprende la schedule lineare è citato separatamente come Shi ed Eberhart 1999 [33, 34]. Per CLPSO, la probabilità di apprendimento è

$$P_{c,i} = 0.05 + 0.45 \frac{\exp(10i/(N-1)) - 1}{\exp(10) - 1}, \qquad i = 0, \ldots, N-1, \tag{A.16}$$

con esemplari selezionati per coordinata, $c = 1.49445$ e refresh gap pari a sette generazioni. La baseline usa lo snapshot congelato e il limite di velocità 0.2 volte la larghezza del dominio.

A.4 SL-PSO: formula implementata e adattamento della popolazione

SL-PSO ordina la popolazione dal peggiore al migliore; ogni particella non migliore può apprendere da dimostratori con fitness superiore. La versione presente nel codice usa la stessa legge di probabilità di Cheng e Jin [37], espressa con un rango zero-based $r = 0, \dots, N-2$:

$$\gamma = \beta \frac{D}{M}, \qquad P_r = \left(1 - \frac{r}{N}\right)^{\eta}, \qquad \eta = \alpha \log\left(\max\left\{1, \left\lceil \frac{D}{M} \right\rceil\right\}\right), \tag{A.17}$$

con $M = 100$, $\alpha = 0.5$, $\beta = 0.01$. Nella notazione originale, che usa il rango one-based $i = 1, \dots, m-1$, la probabilità è $P_i^L = (1 - (i-1)/m)^{\alpha \log\lceil D/M \rceil}$; ponendo $r = i - 1$ e $N = m$ si ottiene esattamente la forma implementata. Il fattore $\max\{1, \cdot\}$ non altera la formula, poiché D, M sono positivi e $\lceil D/M \rceil \geq 1$. Se l'apprendimento viene attivato, per ogni coordinata è selezionato un dimostratore migliore e lo spostamento è

$$\Delta x_{i,j}^{t+1} = r_1 v_{i,j}^t + r_2 (x_{k(i,j),j}^t - x_{i,j}^t) + r_3 \gamma (\bar{x}_j^t - x_{i,j}^t), \tag{A.18}$$

con $r_1, r_2, r_3 \sim U(0,1)$ e un dimostratore selezionato indipendentemente per coordinata fra le particelle migliori. La configurazione mantiene $N = 40$ per la parità FE; la fonte primaria [37] raccomanda invece $m = M + \lfloor D/10 \rfloor$ con $M = 100$. La popolazione fissa è dunque un adattamento esplicito del protocollo, non un errore nella formula di apprendimento. I risultati vanno etichettati come SL-PSO con popolazione fissata, non come riproduzione integrale di tutti i default originali.

A.5 SPSO-2011

La baseline SPSO-2011 costruisce un grafo diretto casuale di informant con $K = 3$ informant più il self. Sia p_i il personal best e l_i il miglior personal best nell'insieme di informant. Il centro di gravità è

$$G_i = x_i + c \frac{p_i + l_i - 2x_i}{3}, \quad c = \frac{1}{2} + \ln 2, \tag{A.19}$$

con il centro a due punti $G_i = x_i + c(p_i - x_i)/2$ nel caso $p_i = l_i$. Il campione x_i' è estratto dall'ipersfera centrata in G_i e di raggio $\|G_i - x_i\|$, con direzione uniforme e raggio uniforme. L'aggiornamento è

$$v_i^{t+1} = \omega v_i^t + x_i' - x_i^t, \qquad x_i^{t+1} = x_i^t + v_i^{t+1}, \qquad \omega = (2 \ln 2)^{-1}. \tag{A.20}$$

Le coordinate sono normalizzate nel box unitario per la costruzione geometrica e riportate al dominio originale. Il grafo viene rigenerato dopo una generazione senza miglioramento del best globale; il confinamento è quello SPSO-2011, non la mappa comune.

A.6 FIPS

La baseline FIPS usa il ring periodico a due vicini, escludendo la particella centrale. L'aggiornamento implementato è

$$v_i^{t+1} = \chi v_i^t + \frac{\chi}{|\mathcal{N}_i|} \sum_{j \in \mathcal{N}_i} R_{ij} \odot (p_j - x_i^t), \quad R_{ij,k} \sim U(0, \phi), \quad \phi = 4.1, \tag{A.21}$$

con $\chi = 2/(\phi - 2 + \sqrt{\phi^2 - 4\phi})$. Il sorgente campiona pesi casuali coordinata per coordinata; la topologia resta fissa per l'intera run. La variante implementata è quindi FIPS su ring di grado due, non la topologia square a quattro vicini spesso considerata nei confronti classici.

A.7 Competitive Swarm Optimizer e DMS-PSO

Nel Competitive Swarm Optimizer (qui abbreviato nel testo come *Competitive Swarm Optimizer*, per evitare collisioni con l'acronimo CPSO), le particelle vengono accoppiate casualmente. Il vincitore resta invariato; il perdente aggiorna

$$\begin{aligned} v_l^{t+1} &= r_1 \odot v_l^t + r_2 \odot (x_w^t - x_l^t) + \varphi_{\mathrm{CSO}} r_3 \odot (\bar{x}^t - x_l^t), \\ x_l^{t+1} &= x_l^t + v_l^{t+1}, \qquad \varphi_{\mathrm{CSO}} = 0.1, \end{aligned} \tag{A.22}$$

con coefficienti casuali vettoriali. Soltanto i perdenti vengono rivalutati e una generazione completa consuma $N/2$ FE se N è pari.

La baseline DMS-PSO usa sottosciami di dimensione obiettivo tre, riorganizzati casualmente ogni cinque generazioni. L'aggiornamento è

$$\begin{aligned} v_i^{t+1} &= \omega_t v_i^t + 2r_1 \odot (p_i - x_i^t) + 2r_2 \odot (l_i - x_i^t), \\ x_i^{t+1} &= x_i^t + v_i^{t+1}, \end{aligned} \tag{A.23}$$

con inerzia lineare da 0.9 a 0.2. La variante di progetto non include ricerca locale quasi-Newton; il resto della partizione è aggiunto all'ultimo sottosciame.

A.8 Controlli da completare prima del benchmark definitivo

Prima delle campagne esterne devono essere completati e archiviati: (i) congelamento esplicito della popolazione SL-PSO a $N = 40$ come adattamento di confronto, con eventuale esperimento separato usando $m = M + \lfloor D/10 \rfloor$; (ii) confronto indipendente delle implementazioni SPSO-2011, FIPS, CSO e DMS-PSO con formule e test di riferimento; (iii) verifica dei limiti $V_{\max}$ registrati nel manifest; (iv) conferma che tutti i metodi eseguano esattamente il budget FE previsto. Le formule di Clerc–Kennedy e CSO sono già coerenti nel codice Python; le anomalie individuate nelle versioni precedenti del LaTeX erano errori redazionali.

B Casualità scalare e limitazione del sottospazio in assenza di correzioni ai vincoli

Questa appendice precisa una conseguenza geometrica della convenzione implementativa adottata nel codice: due scalari casuali vengono estratti per particella e intervallo e condivisi fra tutte le coordinate.

Proposizione B.1 (Invarianza dello span senza correzione ai vincoli)**.** *Sia*

$$\mathcal{S}_0 = \operatorname{span}\{x_1^0, \dots, x_N^0, v_1^0, \dots, v_N^0\} \subseteq \mathbb{R}^D.$$

Se, a ogni passo, i coefficienti della dinamica congelata sono scalari comuni a tutte le coordinate di ciascuna particella, gli attrattori personali e sociali sono selezionati tra le posizioni precedentemente generate e la mappa di vincolo non modifica lo stato, allora

$$x_i^k, v_i^k, p_i^k, s_i^k \in \mathcal{S}_0 \quad \textit{per ogni } i, k, \qquad \dim(\mathcal{S}_0) \le \min(D, 2N).$$

Dimostrazione. La proprietà è vera per x_i^0, v_i^0 per definizione di $\mathcal{S}_0$; anche $p_i^0 = x_i^0$ appartiene allo spazio. Supponiamo che gli stati e i migliori personali al passo k appartengano a $\mathcal{S}_0$. Il leader sociale s_i^k è scelto fra i migliori personali, quindi appartiene allo stesso spazio. Per una particella, q_i^k e α_k sono scalari comuni alle coordinate. Se $q_i^k > 0$, l'equilibrio è

$$x_i^{\mathrm{eq},k} = \frac{a_i^k}{q_i^k} p_i^k + \frac{b_i^k}{q_i^k} s_i^k \in \mathcal{S}_0.$$

La soluzione del sistema lineare congelato è una combinazione lineare di x_i^k, v_i^k e $x_i^{\mathrm{eq},k}$, con coefficienti scalari indipendenti da j; dunque la posizione e la velocità propagate appartengono a $\mathcal{S}_0$. Nel caso $q_i^k = 0$, la soluzione è una combinazione di x_i^k e v_i^k e la conclusione resta valida. Se la mappa di vincolo non modifica lo stato, il passo successivo conserva l'appartenenza. Il personal best viene selezionato fra posizioni già appartenenti a $\mathcal{S}_0$, quindi resta nello spazio. Per induzione la proprietà vale per ogni k. La dimensione è al più il numero dei generatori, $2N$, e non può superare D. □

B.1 Perché il risultato non equivale a irraggiungibilità

La mappa clamp–restitution opera coordinata per coordinata e non è una trasformazione lineare globale; quando è attiva può spostare la posizione fuori da $\mathcal{S}_0$. Pertanto il risultato precedente è una proprietà del tratto non corretto della dinamica, non un vincolo assoluto sulla traiettoria completa. Analogamente, in una singola isola di $n = N/M$ particelle la sottopopolazione locale, prima delle migrazioni e senza correzioni ai vincoli, ha un sottospazio generato da al più $2n$ vettori. Gli immigrati possono introdurre direzioni provenienti da altre isole e ampliare lo span locale.

B.2 Implicazioni per i benchmark presenti

Nella campagna interna principale $N = 64$, quindi $2N = 128$ e il limite dimensionale non forza uno span proprio in $D = 30$ o $D = 100$; in assenza di correzioni, i vettori iniziali possono generare tutto $\mathbb{R}^D$. Nelle campagne esterne, invece, $N = 40$ e $2N = 80$: per $D = 160, 320, 640$ in BBOB-LargeScale e per $D = 1000$ nominali in CEC, il sottospazio non corretto può avere dimensione strettamente inferiore a D. Nel modello a isole principale, $N = 64$ e otto isole comportano $n = 8$, quindi il sottospazio locale iniziale è al più di dimensione 16 prima delle migrazioni.

Questo fatto deve essere considerato una limitazione potenzialmente rilevante, non una prova che l'ottimo sia irraggiungibile. Per quantificarne l'effetto occorre registrare almeno: (i) rango numerico degli stati/posizioni per generazione; (ii) frequenza di attivazione della mappa dei vincoli per coordinata e particella; (iii) rango locale prima e dopo la migrazione; (iv) risultati su problemi traslati e ruotati. Una variante con $r_{1,i,j}^k, r_{2,i,j}^k$ indipendenti per coordinata modificherebbe la formulazione stocastica; potrà essere studiata soltanto come ablation separata e non sarà denominata CPSO di riferimento.

C Dettaglio dei profili sperimentali e delle unità statistiche

C.1 Parametri delle campagne interne

La configurazione architetturale principale definisce $N = 64$, $K = 500$, $D \in \{30, 100\}$, 30 seed per gli stadi di qualità e $\Delta t = 0.35$. I parametri della dinamica sono $c_1 = c_2 = 2.05$, $\mu_{\max} = 0.9$, $\mu_{\min} = 0.4$, $\rho = 0.1$ ed $e = 0.5$. La schedule lineare è indicizzata

sul numero di aggiornamenti pianificati K; le curve anytime a budget intermedio descrivono quindi il prefisso di una schedule parametrizzata dal budget completo, non una nuova schedule ottimizzata per ciascun budget troncato.

Tabella C.13: Configurazioni delle isole nella campagna architetturale principale.

Isole	Worker	Topologia locale	Migrazione	Intervallo
2	2	Global	Nessuna	–
4	4	Global	Nessuna	–
8	8	Global	Nessuna	–
4	4	Global	Ring	5
4	4	Global	Ring	20
4	4	Global	Ring	50
4	4	Global	Fully connected	20
4	4	Ring	Nessuna	–
4	4	Ring	Ring	20

La migrazione è eseguita quando $k + 1$ è multiplo dell'intervallo e non dopo l'ultima generazione. Il codice sostituisce una sola particella peggiore per isola destinataria, solo se l'emigrante è strettamente migliore. La sostituzione aggiorna posizione, personal best e valore corrente, azzera la velocità e mantiene lo stream casuale del destinatario. Le isole richiedono dimensione locale almeno due, divisibilità di N per il numero di isole e worker non superiori al numero di isole; la griglia von Neumann non è supportata intra-isola.

C.2 Screening del passo temporale

Il profilo `configs/campaigns/cpso_dt_sensitivity_m2_screening.json` è esplicitamente esplorativo: usa sei funzioni, $D \in \{30, 100\}$, $N = 64$, $K = 500$ e cinque seed sul MacBook Air M2. La griglia di Δt è

$$\{0.1, 0.2, 0.35, 0.5, 0.7, 1, 1.5, 2.2, 3.6, 5.5, 8, 12, 20, 30, 50\}.$$

Il parametro `reference_dt=30` è un riferimento del grafico di screening, non il valore Δt delle configurazioni architetturali e non dimostra che 30 sia ottimale. Il profilo di campagna corrente usa 0.35. La decisione su Δt dovrà essere congelata dopo aver valutato i profili esplorativi, senza selezionare e testare sullo stesso insieme di seed come se fosse una verifica indipendente. Il riepilogo da compilare è predisposto nell'Appendice D, Tabella D.16 e Figura D.8.

C.3 Profili esterni e carico computazionale

Tabella C.14: Contabilità delle campagne esterne. I conteggi FE sono determinati dalle configurazioni e dalla moltiplicazione tra problemi, istanze, metodi e budget per run.

Suite	Problemi per metodo	Metodi	Run complessive	FE complessive
COCO BBOB	$24 \times 6 \times 15$	11	23 760	3.168 miliardi
COCO BBOB-LargeScale	$24 \times 6 \times 15$	11	23 760	49.896 miliardi
CEC 2013 LSGO	15×25	11	4 125	12.375 miliardi
Totale			51 645	65.439 miliardi

Il conteggio FE totale BBOB-LargeScale è elevato perché combina 24 funzioni, sei dimensioni, 15 istanze, 11 metodi e $10^4 D$ FE per run. La riduzione del budget non è automatica: un pre-pilot deve determinare il costo per FE e verificare se i target anytime siano raggiunti con il budget proposto. Se il budget cambia, la nuova configurazione deve ricevere un identificatore distinto e non può essere aggregata alla campagna corrente.

C.4 Gerarchia dei dati per l'inferenza

Per l'analisi interna il dato primario viene prima aggregato tra seed entro cella funzione-dimensione, poi tra dimensioni entro funzione. Le sei funzioni rimangono le unità di livello superiore. Per BBOB, le 24 funzioni sono le unità di benchmark; istanze e dimensioni sono fattori ripetuti/annidati e non aumentano il numero delle funzioni indipendenti. Per CEC LSGO, le 15 funzioni sono le unità di benchmark e i 25 seed sono repliche entro funzione. Le famiglie dei confronti e i target devono essere pre-specificati; i valori dei seed o delle dimensioni non possono essere utilizzati per gonfiare il numero di osservazioni indipendenti.

Con sei unità, il test signed-rank esatto bilaterale ha $p_{\min} = 0.03125$. Per due o più confronti, il primo valore aggiustato di Holm è almeno 0.0625 anche nel caso limite di sei differenze concordi e non nulle. L'analisi interna non potrà quindi supportare significatività pairwise Holm al livello 0.05; deve restare descrittiva/esplorativa. Le suite esterne aumentano il numero di funzioni, ma la struttura di dipendenza all'interno delle suite deve essere mantenuta nell'analisi.

D Tabelle e figure estese dei risultati

Questa appendice raccoglie i dettagli che appesantirebbero il corpo principale. I risultati interni di asincronia/isole/topologie usano $N = 64$, $K = 200$ e 12 864 FE/run; lo screening di Δt usa $K = 500$ e 32 064 FE/run. Le celle TBD sono intenzionalmente vuote. I grafi e le curve ricavati dai pilot M2 hanno valore esplorativo, non confirmatory.

D.1 Risultati interni per funzione e dimensione

Tabella D.15: Template esteso per risultati interni. Riportare mediana [IQR] del log-errore sui seed, successi e run valide separatamente per ogni funzione-dimensione.

Funzione	D	Metodo	Run valide	Mediana $\log_{10}(f - f^*)$	IQR	Successi	FE/run
Sphere	30	TBD	TBD	TBD	TBD	TBD	12 864
Sphere	100	TBD	TBD	TBD	TBD	TBD	12 864
Rastrigin	30	TBD	TBD	TBD	TBD	TBD	12 864
Rastrigin	100	TBD	TBD	TBD	TBD	TBD	12 864
Rosenbrock	30	TBD	TBD	TBD	TBD	TBD	12 864
Rosenbrock	100	TBD	TBD	TBD	TBD	TBD	12 864
Ackley	30	TBD	TBD	TBD	TBD	TBD	12 864
Ackley	100	TBD	TBD	TBD	TBD	TBD	12 864
Griewank	30	TBD	TBD	TBD	TBD	TBD	12 864
Griewank	100	TBD	TBD	TBD	TBD	TBD	12 864
Salomon	30	TBD	TBD	TBD	TBD	TBD	12 864
Salomon	100	TBD	TBD	TBD	TBD	TBD	12 864

D.2 Sensibilità al passo temporale

Il pilot locale di screening comprende 900 run: sei funzioni, due dimensioni ($D = 30, 100$), 15 valori di Δt e cinque seed appaiati; $N = 64$, $K = 500$ e 32 064 FE/run. La tabella riporta le differenze appaiate rispetto al riferimento grafico $\Delta t = 30$. Tale riferimento non è un optimum dimostrato. Le campagne architetturali, asincrone e topologiche attualmente disponibili usano invece $\Delta t = 0.35$; i due protocolli non devono essere confusi.

Tabella D.16: Template per lo screening di Δt. Riportare mediana [IQR] sui seed entro ciascuna funzione e poi una sintesi tra le sei funzioni, separatamente per dimensione.

Δt	$D = 30$: mediana [IQR]	$D = 100$: mediana [IQR]	FE/run	Run valide	Note
0.10	TBD	TBD	32 064	TBD	TBD
0.20	TBD	TBD	32 064	TBD	TBD
0.35	TBD	TBD	32 064	TBD	TBD
0.50	TBD	TBD	32 064	TBD	TBD
0.70	TBD	TBD	32 064	TBD	TBD
1.00	TBD	TBD	32 064	TBD	TBD
1.50	TBD	TBD	32 064	TBD	TBD
2.20	TBD	TBD	32 064	TBD	TBD
3.60	TBD	TBD	32 064	TBD	TBD
5.50	TBD	TBD	32 064	TBD	TBD
8.00	TBD	TBD	32 064	TBD	TBD
12.00	TBD	TBD	32 064	TBD	TBD
20.00	TBD	TBD	32 064	TBD	TBD
30.00	TBD	TBD	32 064	TBD	TBD
50.00	TBD	TBD	32 064	TBD	TBD

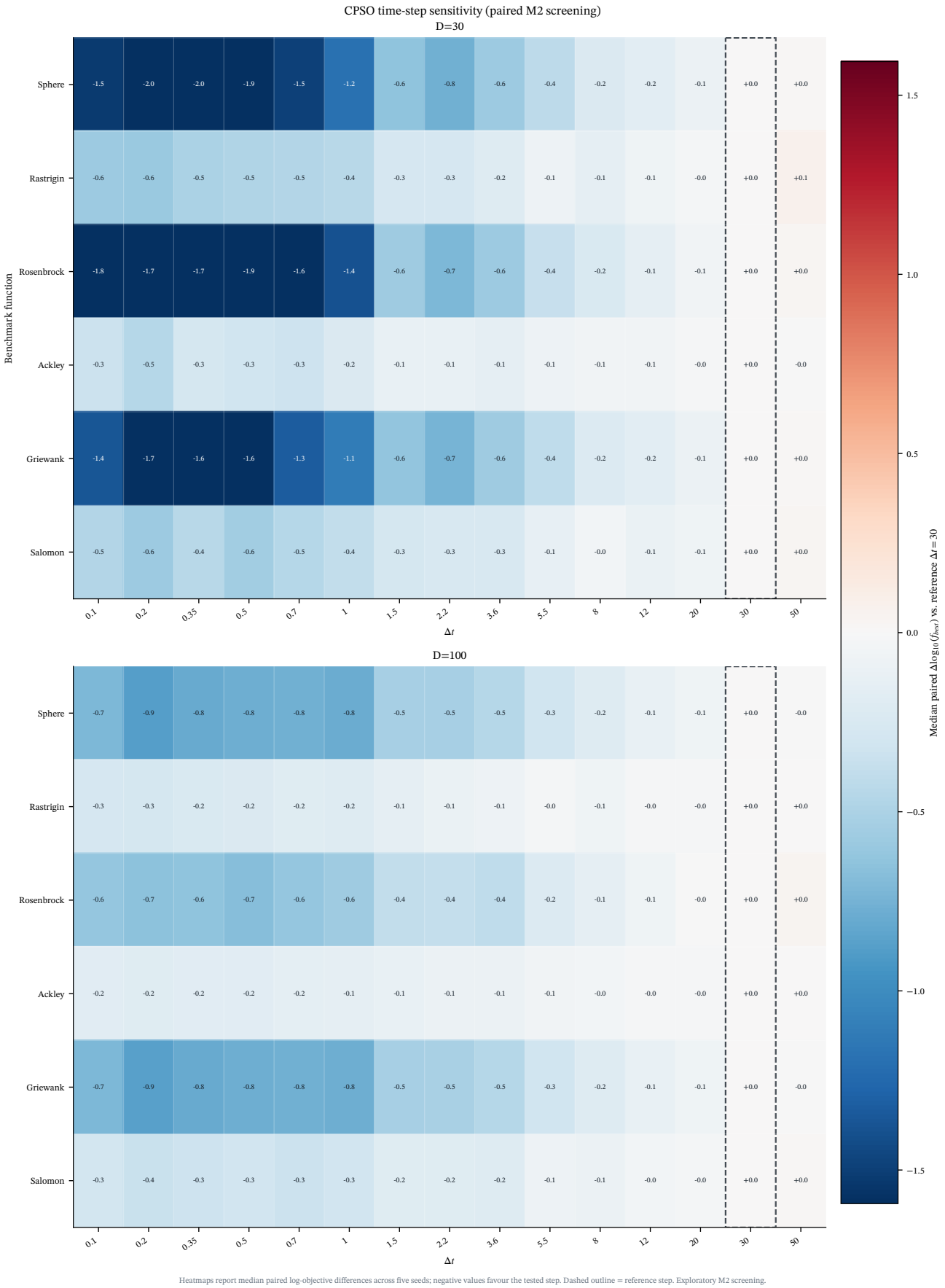


Figura D.8: Sensibilità della qualità della CPSO al passo temporale Δt. Differenze appaiate rispetto al riferimento grafico $\Delta t = 30$, non assunto come optimum. Pilot M2 esplorativo.

D.3 Breakdown dello scaling

Tabella D.17: Template per il breakdown temporale nel pilot `sphere_work`. Sono mostrate le configurazioni seriale e a quattro worker per ciascuna combinazione di dimensione e carico; la figura include anche i casi a uno e due worker. I valori sono medie aritmetiche tra cinque run, così le componenti e il residuo sommano alla media del tempo totale.

D	N	Lavoro	Backend	Worker	Init.	Setup topol.	Propag.	Selez. soc.	Eval.	Residuo	Totale
30	64	512	Serial	1	TBD	TBD	TBD	TBD	TBD	TBD	TBD
30	64	512	Master–slave	4	TBD	TBD	TBD	TBD	TBD	TBD	TBD
30	64	512	Particle-parallel	4	TBD	TBD	TBD	TBD	TBD	TBD	TBD
30	64	4096	Serial	1	TBD	TBD	TBD	TBD	TBD	TBD	TBD
30	64	4096	Master–slave	4	TBD	TBD	TBD	TBD	TBD	TBD	TBD
30	64	4096	Particle-parallel	4	TBD	TBD	TBD	TBD	TBD	TBD	TBD
100	64	512	Serial	1	TBD	TBD	TBD	TBD	TBD	TBD	TBD
100	64	512	Master–slave	4	TBD	TBD	TBD	TBD	TBD	TBD	TBD
100	64	512	Particle-parallel	4	TBD	TBD	TBD	TBD	TBD	TBD	TBD
100	64	4096	Serial	1	TBD	TBD	TBD	TBD	TBD	TBD	TBD
100	64	4096	Master–slave	4	TBD	TBD	TBD	TBD	TBD	TBD	TBD
100	64	4096	Particle-parallel	4	TBD	TBD	TBD	TBD	TBD	TBD	TBD

Figura da generare
`figures/results/fig_runtime_decomposition.pdf`
Decomposizione media in inizializzazione, setup topologico, propagazione, selezione sociale, valutazione obiettivo e residuo non strumentato.

Figura D.9: Decomposizione del wall time medio per il carico sintetico `sphere_work` ($N = 64$, $D = 30, 100$, lavoro 512/4096), separata per backend e numero di worker. Il residuo chiude il bilancio rispetto al tempo totale e non è una misura diretta della sola sincronizzazione. Pilot M2 esplorativo.

D.4 Risultati per topologia e metriche di rete

Tabella D.18: Parametri di costruzione delle topologie dichiarati nel pilot M2. I descrittori del grafo effettivamente realizzato (grado medio e deviazione standard, diametro, cammino medio, clustering e componenti) non sono esportati nei record attuali e non vengono inferiti dalle etichette.

Topologia	Parametri dichiarati nella configurazione
Global-best	Migliore personale globale come informazione sociale
Ring	Anello periodico
Von Neumann	Griglia toroidale 8×8
Wheel	Grafo wheel su $N = 64$ vertici
Random	Parametro di grado configurato pari a 4
Watts–Strogatz	Parametro di grado 4; probabilità di rewiring 0.15
Barabási–Albert	Parametro di grado configurato pari a 4

D.5 Figure supplementari

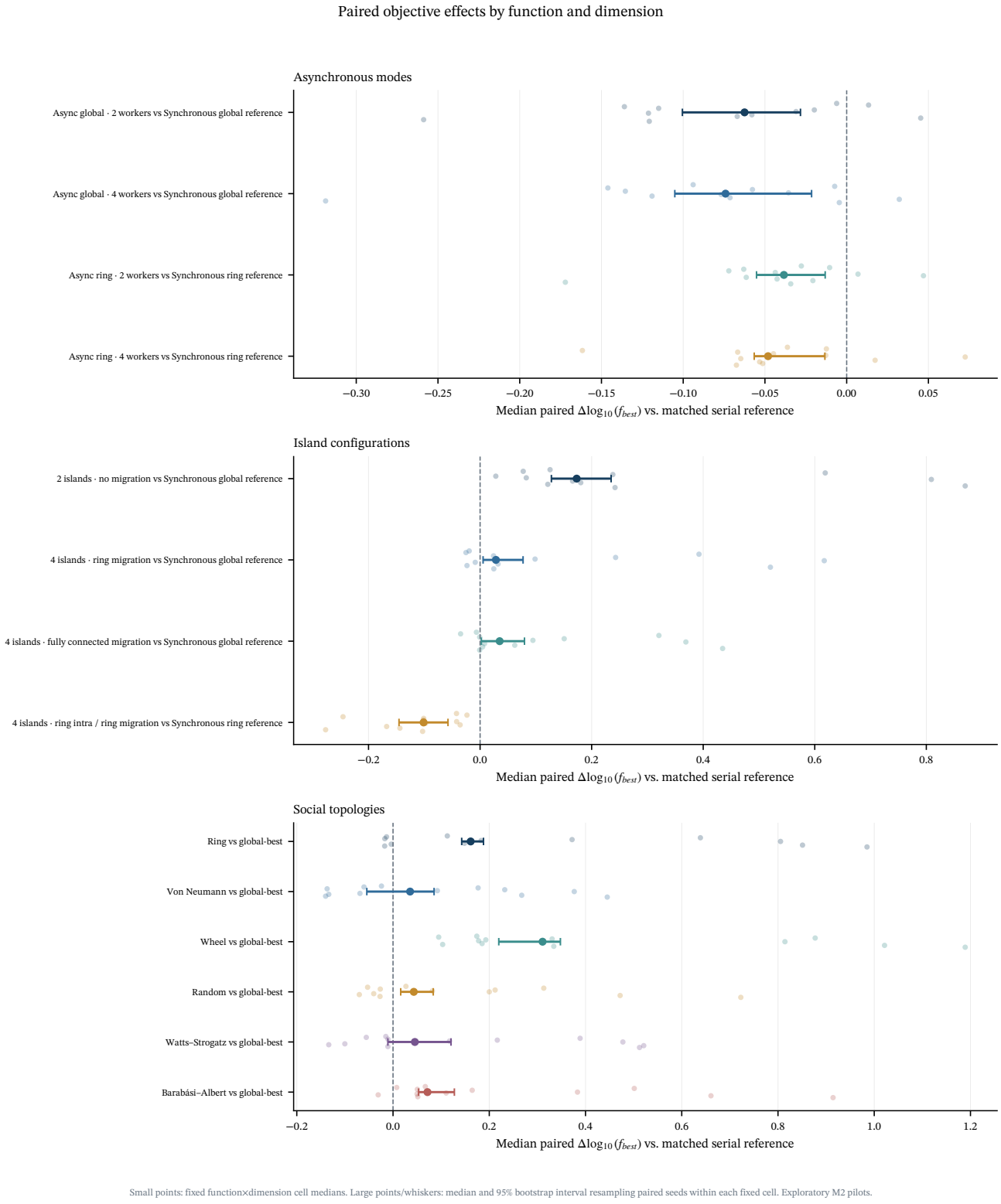


Figura D.10: Breakdown degli effetti appaiati per funzione e dimensione nelle campagne interne. Gli intervalli bootstrap sono condizionati alle sei funzioni e ricampionano i seed appaiati all'interno di ciascuna cella funzione–dimensione. Pilot M2 esplorativo.

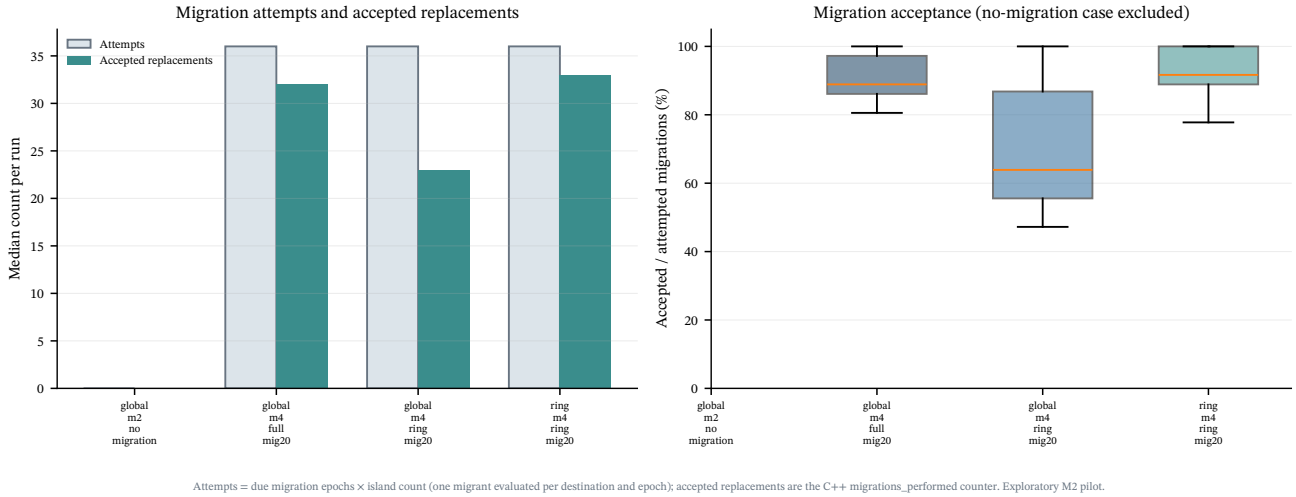
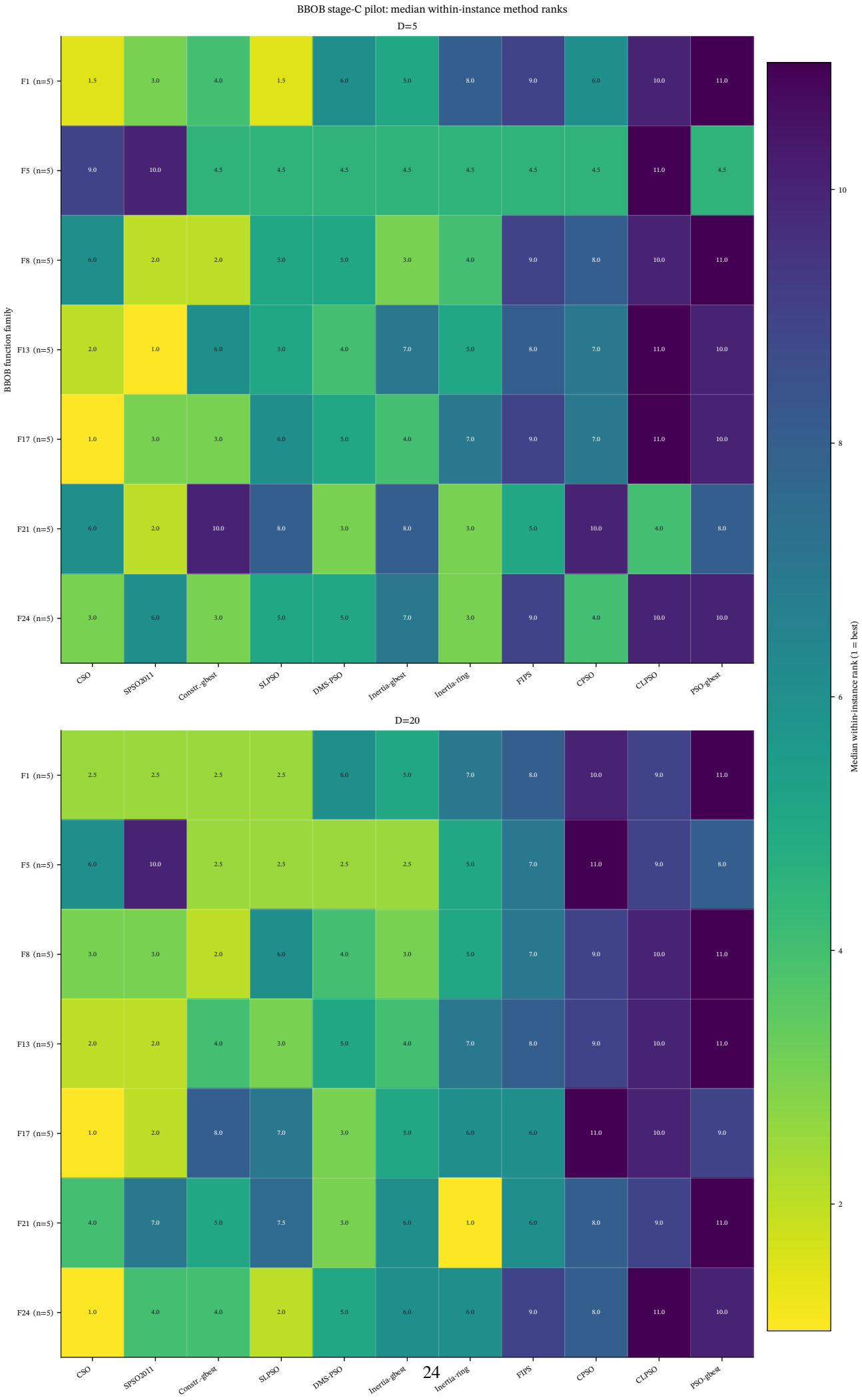


Figura D.11: Diagnostica delle migrazioni nelle configurazioni a isole. I tentativi sono ricostruiti dal calendario deterministico delle epoche; le sostituzioni accettate sono registrate dal motore C++. Pilot M2 esplorativo.


BBOB stage-C pilot: median within-instance method ranks
D=5
F1 (n=5)
F5 (n=5)
F8 (n=5)
F13 (n=5)
F17 (n=5)
F21 (n=5)
F24 (n=5)
BBOB function family
1.5 3.0 4.0 1.5 6.0 5.0 8.0 9.0 6.0 10.0 11.0
9.0 10.0 4.5 4.5 4.5 4.5 4.5 4.5 4.5 11.0 4.5
6.0 2.0 2.0 5.0 5.0 3.0 4.0 9.0 8.0 10.0 11.0
2.0 1.0 6.0 5.0 4.0 7.0 5.0 8.0 7.0 11.0 10.0
1.0 3.0 3.0 6.0 5.0 4.0 7.0 9.0 7.0 11.0 10.0
6.0 2.0 10.0 8.0 3.0 8.0 3.0 5.0 10.0 4.0 8.0
3.0 6.0 3.0 5.0 5.0 7.0 3.0 9.0 4.0 10.0 10.0
CSO
SPSO2011
Constr.-gbest
SLPSO
DMS-PSO
Inertia-gbest
Inertia-ring
FIPS
CPSO
CLPSO
PSO-gbest
D=20
2.5 2.5 2.5 2.5 6.0 5.0 7.0 8.0 10.0 9.0 11.0
6.0 10.0 2.5 2.5 2.5 2.5 5.0 7.0 11.0 9.0 8.0
3.0 3.0 2.0 6.0 4.0 3.0 5.0 7.0 9.0 10.0 11.0
2.0 2.0 4.0 3.0 5.0 4.0 7.0 8.0 9.0 10.0 11.0
1.0 2.0 8.0 7.0 3.0 5.0 6.0 6.0 11.0 10.0 9.0
4.0 7.0 5.0 7.5 3.0 6.0 1.0 6.0 8.0 9.0 11.0
1.0 4.0 4.0 2.0 5.0 6.0 6.0 9.0 8.0 11.0 10.0
Method
Median within-instance rank (1 = best)
2
4
6
8
10

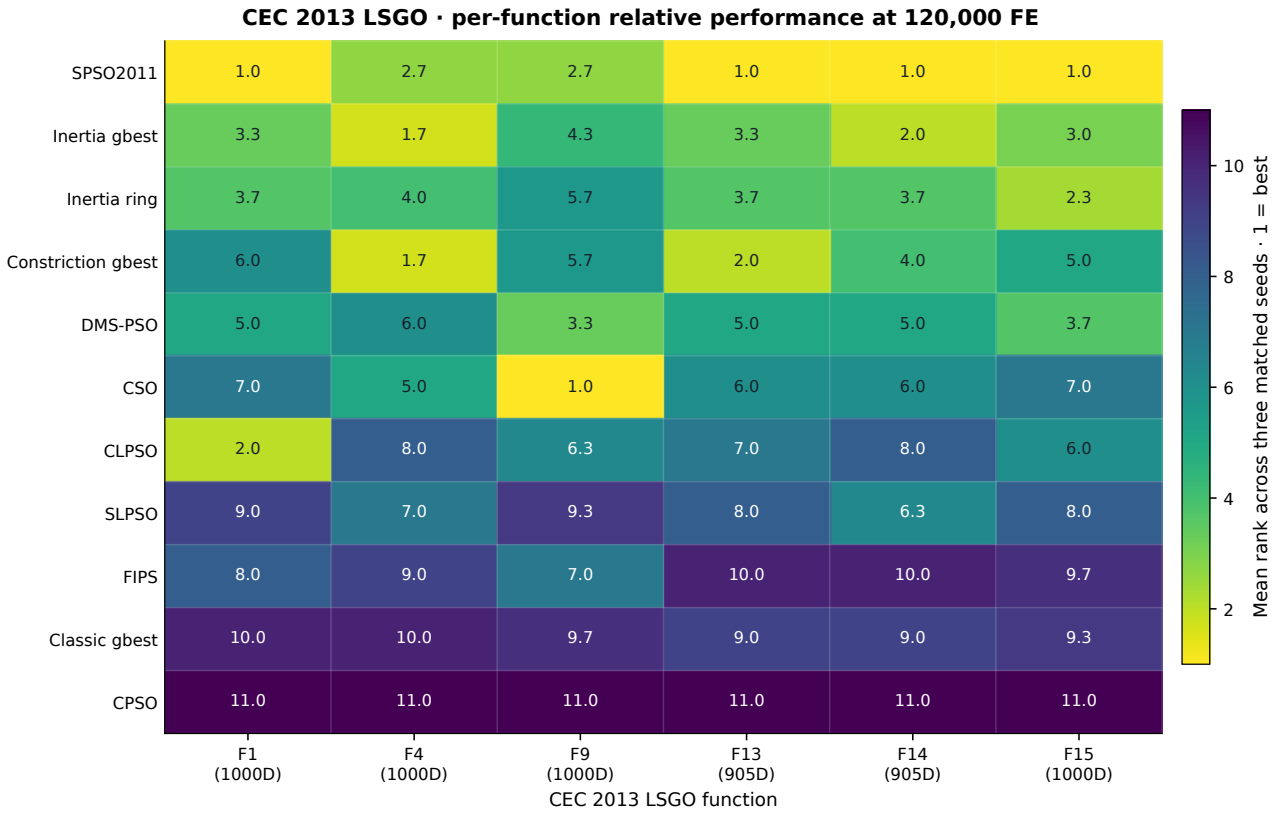


Figura D.13: Breakdown per funzione CEC 2013 LSGO ai checkpoint FE; figura da produrre dopo la campagna CEC e la correzione del driver per F13/F14 a 905 dimensioni.

Riferimenti bibliografici

[1] James Kennedy and Russell Eberhart. Particle swarm optimization. In *Proceedings of ICNN'95 – International Conference on Neural Networks*, volume 4, pages 1942–1948, Perth, Australia, 1995. IEEE. doi: 10.1109/ICNN.1995.488968.

[2] Lingyun Deng and Sanyang Liu. Collective dynamics of particle swarm optimization: A network science perspective. *Physica A: Statistical Mechanics and its Applications*, 675:130778, 2025. doi: 10.1016/j.physa.2025.130778.

[3] Nikolaus Hansen, Anne Auger, Raymond Ros, Olaf Mersmann, Tea Tušar, and Dimo Brockhoff. COCO: a platform for comparing continuous optimizers in a black-box setting. *Optimization Methods and Software*, 36(1):114–144, 2021. doi: 10.1080/10556788.2020.1808977.

[4] Konstantinos Varelas, Ouassim Ait Elhara, Dimo Brockhoff, Nikolaus Hansen, Duc Manh Nguyen, Tea Tušar, and Anne Auger. Benchmarking large-scale continuous optimizers: the bbob-largescale testbed, a COCO software guide and beyond. *Applied Soft Computing*, 97:106737, 2020. doi: 10.1016/j.asoc.2020.106737.

[5] Xiaodong Li, Ke Tang, Mohammad N. Omidvar, Zhenyu Yang, and Kai Qin. Benchmark functions for the CEC'2013 special session and competition on large-scale global optimization. Technical report, Evolutionary Computation and Machine Learning Group, RMIT University, Melbourne, Australia, 2013.

[6] Hassan M. Emara and Hossam A. Abdel Fattah. Continuous swarm optimization technique with stability analysis. In *Proceedings of the 2004 American Control Conference*, pages 2811–2817, Boston, MA, USA, 2004. doi: 10.23919/ACC.2004.1383892.

[7] J. L. Fernández Martínez and E. García Gonzalo. The generalized PSO: A new door to PSO evolution. *Journal of Artificial Evolution and Applications*, 2008:861275, 2008. doi: 10.1155/2008/861275.

[8] J. L. Fernández-Martínez and E. García-Gonzalo. The PSO family: deduction, stochastic analysis and comparison. *Swarm Intelligence*, 3(4):245–273, 2009. doi: 10.1007/s11721-009-0034-8.

[9] J. L. Fernández-Martínez and E. García-Gonzalo. Stochastic stability analysis of the linear continuous and discrete PSO models. *IEEE Transactions on Evolutionary Computation*, 15(3):405–423, 2011. doi: 10.1109/TEVC.2010.2053935.

[10] Mohammad Reza Bonyadi and Zbigniew Michalewicz. Particle swarm optimization for single objective continuous space problems: a review. *Evolutionary Computation*, 25(1):1–54, 2017. doi: 10.1162/EVCO_r_00180.

[11] Sara Grassi and Lorenzo Pareschi. From particle swarm optimization to consensus based optimization: stochastic modeling and mean-field limit. *Mathematical Models and Methods in Applied Sciences*, 31(8):1625–1657, 2021. doi: 10.1142/S0218202521500342.

[12] Calogero Orlando and Angela Ricciardello. Analytic solution of the continuous particle swarm optimization problem. *Optimization Letters*, 15(6):2005–2015, 2021. doi: 10.1007/s11590-020-01671-3.

[13] Enrique Alba and Marco Tomassini. Parallelism and evolutionary algorithms. *IEEE Transactions on Evolutionary Computation*, 6(5):443–462, 2002. doi: 10.1109/TEVC.2002.800880.

[14] Erick Cantú-Paz. *Efficient and Accurate Parallel Genetic Algorithms*, volume 1 of *Genetic Algorithms and Evolutionary Computation*. Kluwer Academic Publishers, 2001. ISBN 978-0-7923-7221-9. doi: 10.1007/978-1-4615-4369-5.

[15] Soniya Lalwani, Harish Sharma, Suresh C. Satapathy, Kusum Deep, and Jagdish C. Bansal. A survey on parallel particle swarm optimization algorithms. *Arabian Journal for Science and Engineering*, 44(4):2899–2923, 2019. doi: 10.1007/s13369-018-03713-6.

[16] Jaco F. Schutte, Jeffrey A. Reinbolt, Benjamin J. Fregly, Raphael T. Haftka, and Alan D. George. Parallel global optimization with the particle swarm algorithm. *International Journal for Numerical Methods in Engineering*, 61(13):2296–2315, 2004. doi: 10.1002/nme.1149.

[17] Byung-Il Koh, Alan D. George, Raphael T. Haftka, and Benjamin J. Fregly. Parallel asynchronous particle swarm optimization. *International Journal for Numerical Methods in Engineering*, 67:578–595, 2006. doi: 10.1002/nme.1646.

[18] Luca Mussi, Fabio Daolio, and Stefano Cagnoni. Evaluation of parallel particle swarm optimization algorithms within the CUDA architecture. *Information Sciences*, 181(20):4642–4657, 2011. doi: 10.1016/j.ins.2010.08.045.

[19] Ming Li, Linhao Huang, Gangyan Xu, and Kong Biao. A parallel particle swarm optimization framework based on a fork-join thread pool using a work-stealing mechanism. *Applied Soft Computing*, 145:110537, 2023. doi: 10.1016/j.asoc.2023.110537.

[20] Vasileios Charilogis, Ioannis G. Tsoulos, and Alexandros Tzallas. An improved parallel particle swarm optimization. *SN Computer Science*, 4:766, 2023. doi: 10.1007/s42979-023-02227-9.

[21] Fahimeh Yazdanpanah and Mohammad Alaei. An approach for low-power heterogeneous parallel implementation of ALC-PSO algorithm using OmpSs and CUDA. *Parallel Computing*, 120:103084, 2024. doi: 10.1016/j.parco.2024.103084.

[22] Mikiko Sato, Mitsuhiko Uchida, and Yuji Sato. Parallelization of MOPSO/D on multi-core CPUs with virtual global best and overlap-aware partitioning. *Artificial Life and Robotics*, 31:536–546, 2026. doi: 10.1007/s10015-025-01093-3. Published online 20 December 2025.

[23] Vandna Rani Verma, Pushkar, Bablu Kumar, Anshul Verma, and Vishnu Sharma. Multicore aware Lyapunov based particle swarm optimization framework for efficient edge task offloading. *Discover Computing*, 29:334, 2026. doi: 10.1007/s10791-026-10224-8.

[24] Konstantinos G. Barkas, Ioannis G. Tsoulos, Vasileios Charilogis, and Glykeria Kyrou. Parallel optimized particle swarm optimization: An efficient parallel approach for neural network training. *Mathematics*, 14(19):3599, 2026. doi: 10.3390/math14193599.

[25] Leonardo Vanneschi, Michele Codecasa, and Giancarlo Mauri. A comparison of parallel and distributed particle swarm optimization algorithms. *New Generation Computing*, 29(2):129–161, 2011. doi: 10.1007/s00354-010-0102-z.

[26] Lucas A. da Silveira, Thaynara A. de Lima, Jessé B. de Barros, José L. Soncco-Álvarez, Carlos H. Llanos, and Mauricio Ayala-Rincón. On the behavior of parallel island models. *Applied Soft Computing*, 148:110880, 2023. doi: 10.1016/j.asoc.2023.110880.

[27] Lucas A. da Silveira, Thaynara A. de Lima, and Mauricio Ayala-Rincón. On reconfiguring heterogeneous parallel island models. *Swarm and Evolutionary Computation*, 89:101624, 2024. doi: 10.1016/j.swevo.2024.101624.

[28] James Kennedy. Small worlds and mega-minds: Effects of neighborhood topology on particle swarm performance. In *Proceedings of the 1999 Congress on Evolutionary Computation*, volume 3, pages 1931–1938. IEEE, 1999. doi: 10.1109/CEC.1999.785509.

[29] James Kennedy and Rui Mendes. Population structure and particle swarm performance. In *Proceedings of the 2002 Congress on Evolutionary Computation (CEC'02)*, volume 2, pages 1671–1676, Honolulu, HI, USA, 2002. IEEE. doi: 10.1109/CEC.2002.1004493.

[30] Duncan J. Watts and Steven H. Strogatz. Collective dynamics of 'small-world' networks. *Nature*, 393(6684):440–442, 1998. doi: 10.1038/30918.

[31] Albert-László Barabási and Réka Albert. Emergence of scaling in random networks. *Science*, 286(5439):509–512, 1999. doi: 10.1126/science.286.5439.509.

[32] Chen Liu, Wen-Bo Du, and Wen-Xu Wang. Particle swarm optimization with scale-free interactions. *PLOS ONE*, 9(5): e97822, 2014. doi: 10.1371/journal.pone.0097822.

[33] Yuhui Shi and Russell C. Eberhart. A modified particle swarm optimizer. In *Proceedings of the 1998 IEEE International Conference on Evolutionary Computation*, pages 69–73, Anchorage, AK, USA, 1998. IEEE. doi: 10.1109/ICEC.1998.699146.

[34] Yuhui Shi and Russell C. Eberhart. Empirical study of particle swarm optimization. In *Proceedings of the 1999 Congress on Evolutionary Computation (CEC'99)*, volume 3, pages 1945–1950, Washington, DC, USA, 1999. IEEE. doi: 10.1109/CEC.1999.785511.

[35] Maurice Clerc and James Kennedy. The particle swarm – explosion, stability, and convergence in a multidimensional complex space. *IEEE Transactions on Evolutionary Computation*, 6(1):58–73, 2002. doi: 10.1109/4235.985692.

[36] Jing J. Liang, A. K. Qin, P. N. Suganthan, and S. Baskar. Comprehensive learning particle swarm optimizer for global optimization of multimodal functions. *IEEE Transactions on Evolutionary Computation*, 10(3):281–295, 2006. doi: 10.1109/TEVC.2005.857610.

[37] Ran Cheng and Yaochu Jin. A social learning particle swarm optimization algorithm for scalable optimization. *Information Sciences*, 291:43–60, 2015. doi: 10.1016/j.ins.2014.08.039.

[38] Mauricio Zambrano-Bigiarini, Maurice Clerc, and Raúl Rojas. Standard particle swarm optimisation 2011 at CEC-2013: A baseline for future PSO improvements. In *Proceedings of the 2013 IEEE Congress on Evolutionary Computation*, pages 2337–2344. IEEE, 2013. doi: 10.1109/CEC.2013.6557848.

[39] Rui Mendes, James Kennedy, and José Neves. The fully informed particle swarm: Simpler, maybe better. *IEEE Transactions on Evolutionary Computation*, 8(3):204–210, 2004. doi: 10.1109/TEVC.2004.826074.

[40] Ran Cheng and Yaochu Jin. A competitive swarm optimizer for large scale optimization. *IEEE Transactions on Cybernetics*, 45(2):191–204, 2015. doi: 10.1109/TCYB.2014.2322602.

[41] Jing J. Liang and P. N. Suganthan. Dynamic multi-swarm particle swarm optimizer. In *Proceedings of the 2005 IEEE Swarm Intelligence Symposium*, pages 124–129. IEEE, 2005. doi: 10.1109/SIS.2005.1501611.

[42] Nikolaus Hansen, Anne Auger, Dimo Brockhoff, and Tea Tušar. Anytime performance assessment in blackbox optimization benchmarking. *IEEE Transactions on Evolutionary Computation*, 26(6):1293–1305, 2022. doi: 10.1109/TEVC.2022.3210897.

[43] Paul Erdős and Alfréd Rényi. On random graphs. i. *Publicationes Mathematicae Debrecen*, 6:290–297, 1959.